

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ КЫРГЫЗСКОЙ РЕСПУБЛИКИ
КЫРГЫЗСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
имени И. РАЗЗАКОВА

Кафедра прикладной математики и информатики

Допустить к защите

Зав. кафедрой _____

«___» _____ 2026 г.

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ

на тему:

**«Разработка программы для расчёта банковского вклада с капитализацией
процентов (на примере ООО «Микрокредитная Компания «Замат
Финанс»»)»**

Направление подготовки: 510200 «Прикладная математика и информатика»

Выполнил: магистрант гр. ПМИ-1-24 _____

Научный руководитель: _____

Рецензент: _____

Бишкек — 2026

СОДЕРЖАНИЕ

[Обновите оглавление в Word: Ссылки → Обновить таблицу → Обновить целиком]

ВВЕДЕНИЕ

Актуальность темы

Рынок микрофинансовых услуг в Кыргызской Республике за последние годы претерпел значительную трансформацию. По данным Национального банка КР, число микрокредитных компаний (МКК) и микрокредитных агентств (МКА) к 2024 году превысило 170 организаций, совокупный кредитный портфель которых достиг 48 млрд сомов. Стремительный рост этого сектора породил высокую конкуренцию, вынуждающую участников рынка расширять линейку финансовых продуктов и внедрять депозитные программы с различными схемами начисления процентов.

Механизм капитализации процентов - присоединение начисленных процентов к основной сумме вклада с последующим начислением процентов уже на увеличенную сумму - математически описан формулами сложных процентов ещё в XVII веке. Однако практическое применение этих формул в условиях реальных финансовых продуктов сопряжено с множеством нюансов: переменные процентные ставки, различные периоды капитализации, досрочное снятие, регулярные пополнения, налоговые удержания. Ручной расчёт таких параметров сопровождается высокой вероятностью ошибок, особенно при обработке десятков клиентских заявок ежедневно.

ООО «Микрокредитная Компания «Замат Финанс», работающая под торговой маркой Doke.kg, предоставляет услуги онлайн-кредитования в Кыргызстане и зарегистрирована Национальным банком КР (свидетельство №517 от 07.12.2022). Компания обслуживает тысячи клиентов, и расчёт параметров вкладов с капитализацией в настоящее время выполняется сотрудниками с использованием электронных таблиц, что не обеспечивает необходимой скорости и надёжности вычислений. Автоматизация этого процесса средствами специализированного программного обеспечения - задача, актуальная как для самой компании, так и для всего сектора МКО республики.

Противоречие между потребностью микрофинансового рынка в точных и быстрых расчётах депозитных продуктов с капитализацией и отсутствием

адаптированных под условия МКО Кыргызстана программных инструментов определяет актуальность настоящего исследования.

Степень разработанности проблемы

Теоретические основы финансовой математики, включая модели сложных процентов и капитализации, представлены в трудах Е.М. Четыркина, А.А. Козлова, В.В. Ковалёва, И.Я. Лукасевича. Вопросы автоматизации банковских вычислений рассматривались в работах Д.А. Ендовицкого, А.И. Уринцева, зарубежных авторов F. Fabozzi, S. Benninga. Математические аспекты дискретного и непрерывного начисления процентов исследовали J.C. Hull, P. Wilmott. Вместе с тем специфика программной реализации расчётов для микрокредитных организаций Кыргызстана, действующих по нормативам НБКР, изучена недостаточно - большинство существующих решений ориентированы на крупные коммерческие банки и не учитывают особенности продуктовой линейки МКО.

Проблема исследования

Проблема исследования состоит в противоречии между необходимостью обеспечения точных, оперативных и прозрачных расчётов вкладов с капитализацией процентов в микрокредитных организациях и недостаточностью доступных программных средств, адаптированных к условиям микрофинансового рынка Кыргызской Республики.

Цель исследования

Цель исследования - разработать и программно реализовать приложение для автоматизированного расчёта банковского вклада с капитализацией процентов, обеспечивающее точность финансовых вычислений и удобство использования сотрудниками микрокредитной компании Doke.kg.

Задачи исследования

- 1) проанализировать теоретические основы финансовой математики в части начисления процентов с капитализацией, изучить действующие модели расчёта вкладов и депозитных продуктов;
- 2) исследовать нормативно-правовую базу Кыргызской Республики,

регулирующую деятельность микрокредитных организаций и порядок начисления процентов по вкладам;

3) провести сравнительный анализ существующих программных средств и онлайн-калькуляторов для расчёта вкладов, выявить их функциональные ограничения;

4) разработать математическую модель расчёта вклада с капитализацией, учитывающую переменные ставки, различные периоды капитализации, пополнение и частичное снятие средств;

5) спроектировать архитектуру программного приложения и реализовать его прототип с графическим пользовательским интерфейсом;

6) выполнить тестирование разработанной программы на контрольных примерах, провести верификацию результатов и оценку точности вычислений;

7) сформулировать рекомендации по внедрению программного продукта в операционную деятельность ООО «Микрокредитная Компания «Замат Финанс».

Объект и предмет исследования

Объект исследования - процессы автоматизации финансовых расчётов в микрокредитных организациях Кыргызской Республики.

Предмет исследования - методы и алгоритмы программного расчёта банковского вклада с капитализацией процентов, а также инструментальные средства их реализации.

Методы исследования

Теоретической основой работы послужили методы анализа научной литературы и нормативных правовых актов Кыргызской Республики. Математическое моделирование применено для формализации расчётов сложных процентов с учётом переменных параметров вклада. Сравнительный анализ использован при исследовании существующих программных аналогов. Проектирование системы выполнено методами объектно-ориентированного анализа с применением нотации UML. Верификация результатов проведена методом контрольных примеров - сопоставлением автоматизированного расчёта с результатами ручных вычислений по эталонным формулам. Функциональное

и модульное тестирование обеспечили проверку корректности работы отдельных компонентов и программного продукта в целом.

Теоретическая и методологическая база

Теоретическая и методологическая база исследования включает труды отечественных и зарубежных учёных в области финансовой математики (Е.М. Четыркин, В.В. Ковалёв, F. Fabozzi, J.C. Hull), банковского дела и микрофинансирования (И.Я. Лукасевич, Д.А. Ендовицкий), проектирования программных систем (Г. Буч, Р. Мартин, Э. Гамма). Нормативно-правовую основу составляют Закон Кыргызской Республики «О микрофинансовых организациях в Кыргызской Республике», положения Национального банка КР о порядке расчёта процентов по вкладам, а также международные стандарты финансовой отчётности (МСФО).

Практическая значимость

Практическая значимость результатов исследования состоит в создании работающего прототипа программы, пригодного для внедрения в операционную деятельность ООО «Микрокредитная Компания «Замат Финанс» (Doke.kg). Разработанный программный продукт может быть адаптирован иными микрокредитными организациями Кыргызстана, а математическая модель расчёта пригодна для использования в учебном процессе при подготовке специалистов по прикладной математике и финансовым вычислениям.

Структура работы

Магистерская диссертация состоит из введения, трёх глав, заключения, списка использованных источников (не менее 30 наименований) и приложений. Во введении обоснована актуальность темы, сформулированы цель, задачи, объект, предмет и методы исследования. Первая глава посвящена теоретическим основам финансовых расчётов и анализу предметной области. Вторая глава содержит описание проектирования программного приложения: от формирования требований до архитектурных решений. Третья глава описывает процесс реализации, тестирования и оценки эффективности разработанного

продукта. Заключение обобщает полученные результаты и формулирует рекомендации по внедрению и развитию системы.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ФИНАНСОВЫХ РАСЧЁТОВ И АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Экономическая сущность банковских вкладов и капитализации процентов

Банковский вклад (депозит) — денежная сумма, которую вкладчик передаёт кредитной организации на определённых условиях ради хранения и получения дохода в виде процентов. По договору банковского вклада кредитная организация принимает поступившие от вкладчика средства, обязуясь возвратить основную сумму и выплатить начисленные проценты в порядке, согласованном сторонами [11]. Депозитные операции относятся к пассивным операциям банка и формируют значительную часть его ресурсной базы. Это их главная функция. Без привлечённых средств клиентов кредитная организация не способна осуществлять активные операции в достаточном объёме.

Институт банковского вклада прошёл длительную эволюцию. Первые формы депозитного хранения зафиксированы ещё в Древнем Вавилоне, где храмы принимали зерно и серебро на хранение, выдавая расписки. Средневековые менялы Италии — банкиры — трансформировали хранительскую функцию в кредитную: привлечённые средства стали размещаться в ссуды, а вкладчикам выплачивался процент как компенсация за временный отказ от пользования деньгами. О.И. Лаврушин прослеживает генезис депозитных операций от примитивного хранения к сложным финансовым продуктам и отмечает, что именно появление срочных вкладов с фиксированной доходностью позволило банкам планировать кредитный портфель [11]. Капитализация процентов — относительно позднее нововведение: массовое распространение вкладов с реинвестированием дохода началось лишь во второй половине XX века, когда автоматизация учётных операций сделала расчёт «процентов на проценты» технически доступным [18].

Е.А. Звонова отмечает, что банковские вклады остаются основным инструментом аккумуляции сбережений населения, несмотря на расширение альтернативных форм инвестирования [23]. Подтверждением этому служит статистика Национального банка Кыргызской Республики: депозитная база

банковского сектора на конец 2024 года составила 592,4 млрд сомов, продемонстрировав прирост на 37,3 % по сравнению с аналогичным периодом предыдущего года [39]. Рост объёмов депозитов зафиксирован по всем категориям вкладчиков. Юридические лица разместили 246,3 млрд сомов, физические лица — 207,7 млрд сомов, а нерезиденты увеличили вложения до 75,4 млрд сомов [39].

Динамика депозитного рынка Кыргызстана за последние годы показывает устойчивый восходящий тренд. Совокупный объём депозитов (банки и микрофинансовые организации) к третьему кварталу 2025 года достиг 715,1 млрд сомов [39]. Депозитный портфель микрофинансового сектора составил 9,8 млрд сомов — прирост на 31 % с начала года. Темпы роста микрофинансовых депозитов опережают банковские, что указывает на растущее доверие вкладчиков к МФО. Для ООО «МКК «Замат Финанс» рост рынка означает увеличение числа депозитных договоров и, как следствие, возросшую потребность в автоматизации расчётных операций.

Классификация банковских вкладов строится на нескольких основаниях. По сроку размещения выделяют вклады до востребования и срочные вклады. Вклад до востребования позволяет вкладчику изъять средства в любой момент без предварительного уведомления, однако процентная ставка по нему минимальна. Срочный вклад предполагает размещение денежных средств на фиксированный период, по истечении которого вкладчик получает основную сумму и проценты. Ставка по срочным вкладам превышает ставку по вкладам до востребования, поскольку банк получает возможность планировать использование привлечённых ресурсов [11]. Помимо указанных двух типов, банковская практика выработала ряд гибридных форм: сберегательные вклады с возможностью пополнения, накопительные вклады с ежемесячным обязательным взносом, а также вклады с капитализацией процентов.

Д.А. Ендовицкий выделяет ещё одно основание для классификации — по категории вкладчика [7]. Вклады физических лиц ориентированы на розничный сегмент и, как правило, предусматривают меньшие минимальные суммы и более простые условия. Вклады юридических лиц сопровождаются повышенными требованиями к документальному оформлению и часто

предлагают индивидуальные ставки, зависящие от суммы и срока. Для МКК «Замат Финанс» классификация по категории вкладчика имеет практическое значение: статус микрокредитной компании ограничивает круг допустимых вкладчиков юридическими лицами и учредителями, что рассмотрено подробно в параграфе 1.3.

По валюте вклады подразделяют на национальные (сомовые — применительно к Кыргызской Республике), валютные и мультивалютные. Не менее существенна классификация по способу начисления процентов. Здесь различают вклады с выплатой процентов в конце срока, с ежемесячной выплатой на отдельный счёт и вклады с капитализацией, при которых начисленные проценты присоединяются к телу вклада [9]. Последний вариант обладает особым экономическим содержанием, раскрытие которого составляет центральную задачу настоящего параграфа.

Процент по банковскому вкладу выступает платой кредитной организации за пользование привлечёнными средствами. Существуют две модели начисления. Механизм расчёта процентов опирается либо на схему простых, либо на схему сложных процентов, и разграничение между ними проведено ещё в классических трудах по финансовой математике [18, 29].

При простом начислении процентный доход рассчитывается исключительно от первоначальной суммы вклада на протяжении всего срока размещения. Формула наращения по простым процентам имеет вид:

$$FV = PV \times (1 + r \times t),$$

где FV — наращенная сумма по окончании срока, PV — первоначальная сумма вклада, r — годовая процентная ставка (в десятичном выражении), t — срок вклада в годах [18]. Линейная зависимость накопленной суммы от времени определяет простоту расчётов, однако ограничивает доходность для вкладчика. При увеличении срока размещения прирост капитала замедляется относительно потенциала сложного процента.

Сложные проценты отличаются тем, что в каждом последующем периоде начисления процентный доход рассчитывается от суммы, увеличенной на ранее начисленные проценты. Е.М. Четыркин определяет этот механизм как

«присоединение процентов к капиталу» [18]. Формула наращенного по сложным процентам с дискретной капитализацией записывается следующим образом:

$$FV = PV \times (1 + r/n)^{(n \times t)},$$

где n — количество периодов капитализации в году [18, 29]. Нелинейная (экспоненциальная) зависимость FV от времени t означает, что прирост вклада ускоряется с каждым новым периодом. Разница между схемами простого и сложного процента становится заметной уже при сроке свыше одного года, а на горизонте 5–10 лет различие приобретает принципиальный характер [19].

Числовой пример позволяет оценить масштаб расхождения на условиях, близких к продуктам Doke.kg. Пусть вкладчик размещает 500 000 сомов под 16 % годовых сроком на 3 года. При простых процентах наращенная сумма составит $FV = 500\,000 \times (1 + 0,16 \times 3) = 740\,000$ сомов, доход — 240 000 сомов. При сложных процентах с ежемесячной капитализацией $FV = 500\,000 \times (1 + 0,16/12)^{36} \approx 804\,329$ сомов, доход — 304 329 сомов. Выигрыш от капитализации составляет 64 329 сомов, то есть 26,8 % дополнительного дохода. При пятилетнем сроке разница возрастает до 42,1 %. Цифры наглядны: для клиента с суммой в полмиллиона сомов выбор между простым и сложным процентом измеряется десятками тысяч сомов.

М.С. Аникин на конкретных числовых примерах продемонстрировал, что при годовой ставке 12 % и ежемесячной капитализации вклад сроком 3 года приносит на 1,95 % больше дохода, чем аналогичный вклад без капитализации. При пятилетнем сроке превышение достигает 3,38 % [19]. Д.В. Мамин подтвердил эту закономерность, рассчитав эффективность вкладов с различными условиями и придя к выводу, что капитализация максимально выгодна при длительных сроках и высоких ставках [24].

Капитализация процентов сводится к реинвестированию начисленного дохода в тело депозита без его фактического изъятия. Вкладчик получает «проценты на проценты» — эффект, который в англоязычной литературе обозначают термином *compounding* [29, 30]. По мнению F.J. Fabozzi, понимание этого механизма составляет фундамент анализа любых инструментов с фиксированным доходом [29].

Частота присоединения начисленных процентов к основной сумме определяет период капитализации. Банковская практика предлагает несколько вариантов: ежегодную, полугодовую, ежеквартальную, ежемесячную и ежедневную капитализацию. Чем выше частота, тем больше итоговый доход вкладчика при прочих равных условиях. Количественную оценку влияния частоты капитализации позволяет провести понятие эффективной процентной ставки, которая вычисляется по формуле:

$$r_{\text{eff}} = (1 + r/n)^n - 1,$$

где r — номинальная годовая ставка, n — число капитализаций за год [18, 9]. Влияние частоты капитализации на эффективную ставку показано в таблице 1.1.1.

Таблица 1.1.1 — Эффективная процентная ставка при номинальной ставке 16 % годовых

Частота капитализации	n	r_eff, %	Доход с 500 000 сомов за 3 года, сомов
Ежегодная	1	16,00	281 216
Полугодовая	2	16,64	293 098
Ежеквартальная	4	16,99	299 427
Ежемесячная	12	17,23	304 329
Ежедневная	365	17,35	306 533

Разница между ежегодной и ежемесячной капитализацией при вкладе 500 000 сомов на 3 года составляет 23 113 сомов — сумма, ощутимая для индивидуального вкладчика. При этом переход от ежемесячного к ежедневному начислению добавляет лишь 2 204 сома, что объясняет предпочтение банков и МКО к ежемесячной капитализации как оптимальному компромиссу между доходностью для клиента и учётной нагрузкой для организации.

Предельным случаем выступает непрерывная капитализация, при которой число периодов n стремится к бесконечности. Формула наращения принимает вид:

$$FV = PV \times e^{(r \times t)},$$

где $e \approx 2,71828$ — основание натурального логарифма [30, 36]. J.C. Hull использует модель непрерывного начисления при оценке производных финансовых инструментов, поскольку она упрощает дифференцирование и интегрирование в аналитических выкладках [30]. Р. Wilmott выводит эту формулу из дифференциального уравнения $dM/dt = r \times M$, где $M(t)$ — сумма на счёте в момент времени t [36]. На практике непрерывная капитализация применяется редко, однако как теоретическая модель она позволяет определить верхнюю границу дохода при заданной номинальной ставке.

С.А. Гамзатова, исследуя депозитные операции коммерческих банков, указывает, что выбор периода капитализации зависит от стратегии привлечения ресурсов: банки, стремящиеся увеличить розничную базу, чаще предлагают ежемесячную капитализацию, а институциональные вкладчики предпочитают вклады с выплатой процентов в конце срока [21]. Для микрокредитных организаций, работающих с небольшими суммами индивидуальных вкладчиков, ежемесячная капитализация служит маркетинговым инструментом: заявленная эффективная ставка выглядит привлекательнее номинальной.

Роль депозитных продуктов в деятельности микрокредитных организаций (МКО) заслуживает отдельного рассмотрения. Закон Кыргызской Республики «О микрофинансовых организациях» наделяет микрофинансовые компании правом привлекать вклады физических и юридических лиц при соблюдении установленных нормативов [2]. По состоянию на октябрь 2025 года на территории Кыргызстана действуют 104 микрофинансовые организации, часть из которых активно размещает депозитные программы [39]. Агентство по защите депозитов КР гарантирует возврат вкладов в коммерческих банках, однако депозиты в МКО под действие системы страхования не подпадают [40]. Отсутствие государственной гарантии повышает требования к прозрачности расчётов и обосновывает потребность в программном инструменте, предоставляющем вкладчику полную детализацию начислений.

Привлечение депозитов позволяет МКО снижать зависимость от внешних заимствований и формировать более дешёвую ресурсную базу, что подробно обосновали А.А. Воротников и А.Ю. Изотова на примере мировых тенденций микрокредитования [20]. Однако депозитная деятельность налагает на МКО

дополнительные обязательства — от соблюдения экономических нормативов НБКР и обеспечения прозрачности расчёта процентов до регулярной отчётности перед регулятором. ООО «Микрокредитная Компания «Замат Финанс» (торговая марка Doke.kg), зарегистрированная НБКР с присвоением свидетельства №517, предлагает депозитные программы с различными вариантами капитализации. Расчёт параметров этих продуктов до настоящего времени выполнялся вручную средствами электронных таблиц, что порождает потребность в специализированном программном инструменте.

Помимо стандартных моделей простого и сложного процента, практика коммерческих организаций требует учёта ряда дополнительных факторов. Среди таких факторов — изменение процентной ставки в течение срока вклада, возможность частичного снятия средств и регулярного пополнения, налоговые удержания. МСФО (IFRS) 9 вводит понятие эффективной процентной ставки (effective interest rate) как метода распределения процентного дохода на период действия финансового инструмента [4]. Применение этого метода обеспечивает корректное отражение доходности вклада в финансовой отчётности, что актуально для МКО, обязанных соблюдать стандарты бухгалтерского учёта.

В.В. Ковалёв, рассматривая финансовые вычисления с позиции аналитика, выделяет два аспекта практического применения формул сложных процентов: прямой расчёт (определение будущей стоимости по известным начальным условиям) и обратный расчёт, или дисконтирование (определение текущей стоимости ожидаемой в будущем суммы) [9, 10]. Оба направления существенны для деятельности Doke.kg. Прямой расчёт позволяет клиенту оценить доход от размещения средств, а дисконтирование используется при формировании внутренней финансовой отчётности компании.

В работе S. Benninga показано, как модели сложного процента реализуются в среде Excel и Python [27]. Ручное применение формул при множестве вариативных параметров неизбежно сопровождается ошибками. Программная автоматизация устраняет этот недостаток и одновременно обеспечивает масштабируемость: одно приложение способно обслуживать сотни клиентских заявок без увеличения трудозатрат.

Модуль расчёта процентов, по оценке Т.В. Полтевой, составляет ядро любой автоматизированной банковской системы; от его корректности зависит достоверность всей финансовой информации организации [25]. Практика подтверждает этот тезис: как показали Р.И. Чен и соавторы, банковский сектор уже перешёл от табличных расчётов к специализированным веб-приложениям, позволяющим клиентам моделировать параметры вклада в режиме реального времени [26].

Проведённый анализ позволяет зафиксировать ряд положений. Банковский вклад по-прежнему остаётся базовым инструментом сбережения; капитализация процентов усиливает его доходность за счёт экспоненциального роста накоплений. Математический аппарат — формулы дискретной и непрерывной капитализации, понятие эффективной процентной ставки, метод эффективной ставки МСФО (IFRS) 9 — проработан глубоко, однако перенос этих моделей на многопараметрические депозитные продукты МКО Кыргызстана без программной автоматизации затруднён. Динамика рынка — рост депозитной базы МФО на 31 % за девять месяцев 2025 года — лишь усиливает потребность в таком инструменте. Именно его проектированию посвящены вторая и третья главы работы.

1.2 Математические модели расчёта сложных процентов с различными режимами капитализации

Депозитные продукты микрокредитных организаций отличаются многообразием условий: ставки меняются по периодам, вкладчики пополняют счёт и снимают часть средств, а государство удерживает налог с процентного дохода. Простейшая формула сложных процентов, рассмотренная в предыдущем параграфе, не охватывает такие сценарии. Построение расчётного программного модуля требует формализации каждого из них в виде отдельной математической модели, пригодной для алгоритмической реализации [18]. Настоящий параграф последовательно расширяет базовую модель, добавляя один дополнительный фактор на каждом шаге, и сопоставляет полученные модели по точности и области применения.

Базовая модель дискретной капитализации

Формула наращенной суммы при фиксированной ставке и равномерных периодах капитализации уже была приведена в §1.1. Здесь она рассматривается подробнее с позиции вычислительной реализации. Запись имеет вид:

$$FV = PV \times (1 + r/n)^{(n \times t)},$$

где PV обозначает начальную сумму вклада, r — номинальную годовую процентную ставку, n — число периодов капитализации за год, t — срок размещения в годах [18]. Выражение $(1 + r/n)$ принято называть множителем наращенной суммы за один период, а показатель степени $n \times t$ равен полному числу начислений за весь срок вклада [9].

Е.М. Четыркин обращает внимание на то, что при программной реализации удобнее разделять расчёт на итерации: на каждом шаге k текущая сумма S_k вычисляется рекуррентно как $S_k = S_{k-1} \times (1 + r/n)$ [18]. Итерационная форма допускает промежуточные проверки, отладку и протоколирование результатов, что упрощает верификацию. Замкнутая формула остаётся эквивалентной, однако при большом числе периодов она подвержена ошибкам округления с плавающей запятой [27]. S. Benninga рекомендует использовать числовые типы повышенной точности (decimal) при финансовых вычислениях, чтобы избежать накопления погрешности [27].

В.В. Ковалёв подчёркивает, что модель дискретной капитализации предполагает постоянство ставки, отсутствие промежуточных денежных потоков и неизменность суммы между начислениями [9]. Три ограничения задают рамки применимости. Снятие любого из них приводит к расширенным моделям.

Рассмотрим числовой пример, приближенный к условиям Doke.kg. Вклад 300 000 сомов размещён на 12 месяцев под 18 % годовых с ежемесячной капитализацией. Множитель наращенной суммы за один месяц: $(1 + 0,18/12) = 1,015$. Число периодов: 12. Наращенная сумма: $FV = 300\,000 \times 1,015^{12} = 300\,000 \times 1,19562 \approx 358\,685$ сомов. Процентный доход: 58 685 сомов. При ежеквартальной капитализации ($n = 4$) множитель за квартал составляет $(1 + 0,18/4) = 1,045$, число периодов 4, и $FV = 300\,000 \times 1,045^4 \approx 357\,813$ сомов. Разница в 872 сома

наглядно показывает эффект частоты капитализации даже на коротком горизонте.

Модель непрерывного начисления

Если частота капитализации n устремляется к бесконечности, дискретная формула переходит в предел:

$$FV = PV \times e^{(r \times t)},$$

где $e \approx 2,71828$ — основание натурального логарифма [30]. Переход обосновывается классическим пределом $\lim_{n \rightarrow \infty} (1 + r/n)^n = e^r$, описанным ещё Эйлером. J.C. Hull выводит эту модель из дифференциального уравнения $dS/dt = r \times S$ с начальным условием $S(0) = PV$, решение которого даёт экспоненту [30]. Р. Wilmott применяет ту же конструкцию при моделировании стоимости безрисковых облигаций [36].

Непрерывная модель удобна в теории. Она позволяет переходить от сложения дискретных приращений к интегрированию, что упрощает аналитические выкладки при оценке производных инструментов [30]. Однако ни один банк и ни одна микрокредитная организация не начисляют проценты непрерывно: бухгалтерский учёт требует фиксированных дат начисления [4]. Поэтому в разрабатываемом приложении непрерывная модель будет реализована как справочный инструмент для определения верхней границы дохода. Для упомянутого выше примера (300 000 сомов, 18 % годовых, 1 год) непрерывная капитализация даёт $FV = 300\,000 \times e^{0,18} \approx 359\,119$ сомов. Разница с ежемесячным начислением — 434 сома, или 0,12 %, что подтверждает практическую достаточность дискретной модели с коротким периодом капитализации [29].

Модель с переменной процентной ставкой

Реальные депозитные договоры редко предусматривают единую ставку на весь срок. ООО «МКК «Замат Финанс» предлагает продукты, в которых ставка зависит от суммы остатка или меняется по временным интервалам. И.Я. Лукасевич формализует расчёт наращения при ставке, меняющейся по периодам, следующим образом [13]:

$$FV = PV \times \prod_{j=1}^m (1 + r_j / n_j)^{(n_j \times t_j)},$$

где m обозначает количество интервалов с разными ставками, r_j — годовую ставку на j -м интервале, n_j — частоту капитализации на j -м интервале, t_j — длительность j -го интервала в годах [13]. Символ $\prod$ означает произведение множителей наращения по всем интервалам.

На практике произведение раскладывается в последовательные умножения. Сначала начальная сумма наращивается за первый интервал по ставке r_1 , затем результат выступает начальной суммой для второго интервала со ставкой r_2 и так далее [18]. Четыркин поясняет, что метод «цепного наращения» эквивалентен общей формуле, но нагляднее для вычислительных алгоритмов [18]. Модель допускает произвольное число интервалов. Это свойство существенно для программного модуля, поскольку Doke.kg может изменить условия продукта, и система должна обрабатывать любую конфигурацию ставок без модификации кода.

Проиллюстрируем на примере. Вкладчик размещает 400 000 сомов на 24 месяца. Первые 12 месяцев ставка составляет 20 % годовых, следующие 12 — 16 %. Капитализация ежемесячная. За первый интервал: $S_{12} = 400\,000 \times (1 + 0,20/12)^{12} \approx 488\,130$ сомов. За второй интервал: $FV = 488\,130 \times (1 + 0,16/12)^{12} \approx 572\,603$ сомов. Совокупный доход: 172 603 сома. Если бы ставка была единой (18 % — среднее арифметическое), результат составил бы $FV \approx 572\,036$ сомов. Расхождение в 567 сомов объясняется тем, что более высокая ставка действовала на первом интервале, когда база начисления меньше. Среднее арифметическое ставок не может заменить пошаговый расчёт.

Модель с регулярными пополнениями (аннуитетная)

Накопительные вклады предполагают, что клиент вносит дополнительные суммы через равные промежутки времени. Ковалёв описывает эту задачу через модель обычного аннуитета, в которой периодические платежи PMT поступают в конце каждого периода [9]. Формула наращения принимает вид:

$$FV = PV \times (1 + i)^N + PMT \times [((1 + i)^N - 1) / i],$$

где $i = r/n$ — ставка за один период начисления, $N = n \times t$ — общее число периодов, PMT — сумма регулярного пополнения [9, 29]. Первое слагаемое отражает рост первоначального взноса по модели сложных процентов, а второе описывает накопленную стоимость потока пополнений.

F.J. Fabozzi уточняет, что формула справедлива при условии совпадения момента пополнения с моментом капитализации [29]. Если клиент вносит средства в начале периода, а не в конце, множитель второго слагаемого корректируется: оно умножается на $(1 + i)$, и модель превращается в аннуитет пренумерандо [18]. Лукасевич рекомендует при проектировании программного обеспечения предоставлять пользователю выбор между двумя вариантами, поскольку различие в итоговой сумме может достигать нескольких процентов при длительных сроках [13].

Пример иллюстрирует масштаб эффекта. При начальном взносе 100 000 сомов, ежемесячном пополнении 5 000 сомов, ставке 14 % годовых и ежемесячной капитализации за 3 года ($N = 36$ периодов) расчёт по формуле даёт: первое слагаемое $PV \times (1 + 0,14/12)^{36} \approx 151\,259$ сомов, второе слагаемое $PMT \times [((1 + 0,14/12)^{36} - 1) / (0,14/12)] \approx 219\,453$ сомов. Итого $FV \approx 370\,712$ сомов. Без капитализации (простые проценты) та же комбинация начального взноса и пополнений дала бы около 352 000 сомов. Выигрыш от капитализации составляет порядка 18 700 сомов, или 5,3 % от суммы без капитализации [24].

Модель с учётом частичных снятий

Вклады ряда депозитных программ Doke.kg допускают частичное изъятие средств без расторжения договора. Математическая модель строится на уменьшении базы начисления в момент снятия [10]. Если в момент времени t (измеренном в периодах от начала вклада) вкладчик изымает сумму W , то расчёт делится на два этапа. До момента t сумма наращивается по стандартной формуле. Непосредственно после снятия база уменьшается, и дальнейшее наращение производится от нового остатка:

$$FV = (S_t - W) \times (1 + i)^{(N - t)},$$

где $S_t = PV \times (1 + i)^t$ — сумма на счёте к моменту снятия, W — величина изъятия, $(N - t)$ — оставшееся число периодов [10]. При нескольких

снятиях формула обобщается по тому же принципу: после каждого изъятия расчёт «перезапускается» от уменьшенной базы.

Частичные снятия снижают эффективную доходность нелинейно. Как показал Д.В. Мамин, изъятие на раннем этапе наносит больший ущерб итоговой сумме, чем аналогичное снятие ближе к завершению срока, поскольку ранние средства имели больший горизонт для начисления процентов [24]. Рассмотрим конкретный пример. Вклад 500 000 сомов, 16 % годовых, 24 месяца, ежемесячная капитализация. Без снятий: $FV = 500\,000 \times 1,01333^{24} \approx 687\,544$ сома. Если на 6-м месяце вкладчик снимает 100 000 сомов, баланс к этому моменту составляет $S_6 \approx 541\,022$, после снятия — 441 022 сома. За оставшиеся 18 месяцев: $FV = 441\,022 \times 1,01333^{18} \approx 554\,673$ сома. Потеря по сравнению с вариантом без снятия — 132 871 сом, что значительно превышает изъятие 100 000 сомов из-за утраченных «процентов на проценты». Программная реализация обязана учитывать произвольное число снятий с произвольными суммами и датами, что исключает использование замкнутой формулы и требует пошагового (итерационного) расчёта.

Комбинированная модель: пополнения, снятия и переменная ставка

Реальный депозит объединяет все перечисленные факторы. Итерационный алгоритм, который реализуется в программном модуле, вычисляет баланс на каждом периоде k по рекуррентному соотношению:

$$S_k = (S_{k-1} + D_k - W_k) \times (1 + r_k / n_k),$$

где D_k — сумма пополнения в периоде k (ноль при отсутствии), W_k — сумма снятия в периоде k (ноль при отсутствии), r_k — годовая ставка, действующая в периоде k , n_k — частота капитализации [18, 9]. Начальное условие: $S_0 = PV$. Итоговая наращенная сумма определяется после завершения последнего периода: $FV = S_N$.

Benninga подчёркивает, что рекуррентный подход универсален: он не требует аналитического замкнутого решения и одинаково корректно обрабатывает любую комбинацию переменных ставок, нерегулярных пополнений и снятий [27]. Вычислительная сложность алгоритма

пропорциональна числу периодов ($O(N)$), что обеспечивает мгновенный отклик даже при посуточном расчёте на горизонте десятков лет.

Налоговый корректор

Законодательство Кыргызской Республики предусматривает налогообложение процентного дохода по вкладам. Ставка налога определяется нормативными актами и может изменяться. Налоговый корректор вводится для определения чистой (после удержания налога) доходности вклада [10]. Формула расчёта чистого дохода:

$$FV_{net} = FV - tax \times (FV - PV),$$

где FV — наращенная сумма до вычета налога, PV — первоначальный вклад, tax — ставка налога на процентный доход в десятичном выражении [10]. Величина $(FV - PV)$ представляет совокупный процентный доход за весь срок. При итерационном расчёте налог может удерживаться единовременно при закрытии вклада либо ежепериодно, в момент капитализации. Второй вариант снижает базу начисления и приводит к меньшей итоговой сумме. Ковалёв описывает оба сценария и рекомендует реализовывать в программном обеспечении оба варианта, предоставляя пользователю выбор [9].

Для случая ежепериодного удержания рекуррентная формула модифицируется:

$$S_k = S_{\{k-1\}} + I_k \times (1 - tax),$$

где $I_k = S_{\{k-1\}} \times (r_k / n_k)$ — начисленные проценты за период k [9]. К основной сумме прибавляется лишь чистый (после налога) процентный доход, что снижает темп наращивания. МСФО (IFRS) 9 требует раскрытия эффективной процентной ставки с учётом всех существенных денежных потоков, включая налоговые удержания, при составлении финансовой отчётности микрофинансовых организаций [4].

Возвращаясь к примеру с вкладом 300 000 сомов под 18 % на 12 месяцев: без налога процентный доход составляет 58 685 сомов. При единовременном удержании 10 % налог равен 5 869 сомов, чистый доход — 52 816 сомов. При ежемесячном удержании итоговая сумма оказывается на 312 сомов меньше из-за сокращённой базы капитализации. Разница невелика на коротком горизонте,

но при сроке 5 лет расхождение между двумя схемами удержания достигает нескольких тысяч сомов.

Сравнительная характеристика моделей

Рассмотренные модели образуют иерархию. Сложность и точность возрастают от базовой формулы к комбинированному алгоритму. Сводная характеристика приведена в таблице 1.2.1.

Таблица 1.2.1 — Сравнительная характеристика математических моделей расчёта вкладов

Модель	Входные параметры	Учёт пополнений	Учёт снятий	Переменная ставка	Вычислит. сложность
Базовая дискретная	PV, r, n, t	Нет	Нет	Нет	O(1)
Непрерывная	PV, r, t	Нет	Нет	Нет	O(1)
Переменная ставка	PV, {r _j , t _j }	Нет	Нет	Да	O(m)
Аннуитетная	PV, r, n, t, PMT	Да (регулярные)	Нет	Нет	O(1)
С частичными снятиями	PV, r, n, t, {W _τ }	Нет	Да	Нет	O(N)
Комбинированная	PV, {r _k }, {D _k }, {W _k }	Да (произвольные)	Да	Да	O(N)

Модель дискретной капитализации при фиксированной ставке пригодна для вкладов с простыми условиями, когда ставка неизменна, движений по счёту нет, а налог не удерживается. Модель непрерывного начисления выполняет роль теоретического ориентира и верхней границы доходности, но не применяется в реальном бухгалтерском учёте [4]. Модель с переменной ставкой расширяет область применения на ступенчатые и прогрессивные депозитные продукты, что характерно для предложений Doke.kg [13]. Аннуитетная модель с пополнениями описывает накопительные программы, а модель с частичными снятиями охватывает гибкие вклады с правом частичного изъятия.

Комбинированная рекуррентная модель объединяет все факторы и допускает произвольную конфигурацию входных параметров.

Точность базовой модели ограничена тремя названными допущениями о постоянстве ставки, отсутствии потоков и отсутствии налога. М.С. Аникин показал, что даже при фиксированных условиях расхождение между дискретной и непрерывной моделями не превышает десятых долей процента на горизонте до 5 лет [19]. Расхождение между замкнутой формулой и итерационным расчётом вызвано исключительно ошибками округления и при использовании числового типа `decimal` устраняется полностью [27]. Комбинированная рекуррентная модель обеспечивает погрешность, определяемую только точностью входных параметров (ставки и суммы), и не вносит систематической ошибки [18].

С позиции вычислительной реализации базовая и аннуитетная модели допускают расчёт по замкнутым формулам за постоянное время $O(1)$, тогда как комбинированная модель требует прохода по всем периодам, что соответствует линейной сложности $O(N)$. При типичных параметрах вкладов Doke.kg (срок до 5 лет, ежемесячная капитализация, $N \leq 60$) вычисление занимает микросекунды и не создаёт нагрузки на сервер.

Практическая значимость выбора модели зависит от конкретного депозитного продукта. Для стандартного вклада Doke.kg сроком 12 месяцев с фиксированной ставкой 16 % годовых и ежемесячной капитализацией базовая дискретная модель даёт точный результат. Однако тот же вклад с возможностью ежемесячного пополнения на 10 000 сомов требует аннуитетной модели. Если вкладчик дополнительно запрашивает частичное снятие через 6 месяцев, аннуитетная модель уже неприменима — необходим переход к комбинированной рекуррентной формуле. А при привязке ставки к учётной ставке НБКР (которая пересматривается 8 раз в год) требуется модель переменной ставки. Таким образом, даже один депозитный продукт при различных конфигурациях параметров может потребовать любую из шести описанных моделей.

Четыркин отмечает, что наращение при переменных ставках и нерегулярных потоках не поддаётся адекватному описанию замкнутой

аналитической формулой [18]. Единственный универсальный подход — пошаговое (итерационное) вычисление, при котором баланс пересчитывается в конце каждого периода капитализации с учётом всех операций, совершённых за этот период. Именно этот подход положен в основу комбинированной рекуррентной модели, и именно его реализует проектируемый программный модуль.

Выбор комбинированной модели как основного алгоритма не исключает использования остальных пяти. Каждая из них выступает частным случаем общей рекуррентной формулы. При отсутствии пополнений $D_k = 0$ для всех k , при отсутствии снятий $W_k = 0$, при постоянной ставке $r_k = r$. Программный модуль передаёт параметры в единый алгоритм, который автоматически вырождается до нужного частного случая. Ковалёв определяет такой подход как принцип обобщённой модели, при которой частные формулы не программируются отдельно, а получаются из общей при подстановке нулевых значений для неиспользуемых параметров [9].

Проведённый анализ математических моделей позволяет сформулировать основу для проектирования вычислительного ядра приложения. Базовая формула дискретной капитализации, модели переменной ставки, аннуитетных пополнений и частичных снятий, а также налоговый корректор составляют минимальный набор, достаточный для автоматизации расчётов ООО «МКК «Замат Финанс». Программный модуль будет использовать комбинированную рекуррентную модель как универсальный алгоритм, способный обрабатывать все типы депозитных продуктов компании. Конкретные архитектурные и проектные решения по реализации этого модуля рассматриваются во второй главе.

1.3 Нормативно-правовое регулирование деятельности микрокредитных организаций в Кыргызской Республике

Программный продукт, разрабатываемый для ООО «МКК «Замат Финанс», функционирует не в вакууме: каждая операция по начислению процентов, формированию отчёта и раскрытию условий вклада подчинена нормативным требованиям. Игнорирование регуляторного контекста при

проектировании финансового приложения ведёт к тому, что формально корректная программа оказывается юридически непригодной. Настоящий параграф анализирует правовую базу, определяющую рамки деятельности микрокредитных организаций (МКО) Кыргызской Республики, с акцентом на те нормы, которые непосредственно затрагивают программную реализацию расчёта вкладов с капитализацией.

Базовым законодательным актом выступает Закон Кыргызской Республики «О микрофинансовых организациях в Кыргызской Республике» от 23 июля 2002 года № 124 (в редакции 2024 года). Закон определяет три организационно-правовые формы микрофинансовой деятельности: микрофинансовую компанию (МФК), микрокредитную компанию (МКК) и микрокредитное агентство (МКА). Разграничение между формами проведено по объёму допустимых операций. Различия принципиальны. МФК обладает правом привлекать вклады физических и юридических лиц, выдавать микрокредиты и осуществлять переводы. МКК вправе привлекать вклады юридических лиц и своих учредителей, а также выдавать микрокредиты, однако ограничена в привлечении средств физических лиц, не входящих в число учредителей. МКА осуществляет исключительно кредитование за счёт донорских средств и грантов, привлечение вкладов ему запрещено [2].

Помимо трёх традиционных форм, законодательство 2024 года закрепило право МФО осуществлять деятельность по исламским принципам финансирования (ИПФ). По состоянию на октябрь 2025 года на территории Кыргызстана действуют 104 микрофинансовые организации, из которых 10 работают на основе ИПФ [39]. Расширение перечня организационных форм увеличивает число параметров, которые программный продукт обязан учитывать при расчёте: продукты ИПФ оперируют понятиями «мурабаха» и «мудароба» вместо классических процентных ставок, что требует отдельной логики начисления дохода.

ООО «МКК «Замат Финанс» зарегистрировано в реестре НБКР под свидетельством № 517 от 07.12.2022 и обладает статусом микрокредитной компании. Этот статус определяет круг клиентов, от которых компания вправе привлекать вклады: юридические лица и учредители. Ограничение влияет на

проектируемое приложение напрямую. При создании карточки вкладчика система обязана верифицировать принадлежность лица к допустимой категории, а попытка оформления вклада от стороннего физического лица блокируется на уровне бизнес-логики.

Регуляторные полномочия в отношении МКО сосредоточены в Национальном банке Кыргызской Республики (НБКР). Статья 13 Закона «О Национальном банке Кыргызской Республики» [3] относит к его компетенции лицензирование, пруденциальный надзор, установление обязательных нормативов и проведение проверок. НБКР утверждает порядок расчёта и раскрытия эффективной процентной ставки, стандарты отчётности и формы бухгалтерских документов. Цена несоблюдения высока. Нарушение нормативов влечёт санкции от письменного предупреждения до отзыва лицензии [39].

Пруденциальные нормативы НБКР для МКО включают минимальный размер уставного капитала, нормативы достаточности капитала и ограничения на концентрацию рисков. Минимальный уставной капитал МКК установлен на уровне 5 млн сомов. Для МФК порог выше — 50 млн сомов, что отражает расширенный круг допустимых операций [39]. МКО обязаны ежеквартально представлять финансовую отчётность регулятору и проходить внешний аудит не реже одного раза в год. Отчётные формы включают баланс, отчёт о прибылях и убытках, расчёт экономических нормативов и расшифровку депозитного портфеля по срокам и ставкам. Программное обеспечение, автоматизирующее расчёт вкладов, обязано генерировать данные, пригодные для заполнения этих форм без ручной обработки.

Постановление Правления НБКР № 2019-П-12/57-4 «Об утверждении Положения о порядке расчёта эффективной процентной ставки по депозитам и кредитам» устанавливает методику вычисления эффективной процентной ставки (ЭПС). ЭПС определяется как ставка дисконтирования, при которой приведённая стоимость всех денежных потоков по вкладу равна первоначальной сумме:

$$PV = \sum (CF_k / (1 + ЭПС/n)^k) = 0,$$

где CF_k — денежный поток в периоде k , n — число периодов капитализации в году. Методика обязывает кредитную организацию раскрывать ЭПС при заключении договора вклада и указывать её в договоре явным образом [39]. Требование означает, что проектируемое приложение не может ограничиться расчётом номинальной суммы вклада: модуль расчёта обязан дополнительно вычислять ЭПС и предъявлять её оператору.

Методика НБКР по расчёту ЭПС согласуется с требованиями МСФО (IFRS) 9 «Финансовые инструменты», который Кыргызстан применяет с 2018 года. Стандарт вводит метод эффективной процентной ставки как основной способ оценки финансовых активов по амортизированной стоимости. Процентный доход рассчитывается умножением ЭПС на валовую балансовую стоимость финансового актива [4]. Для депозитных продуктов с постоянной ставкой и равномерной периодичностью капитализации ЭПС совпадает с эффективной годовой ставкой r_{eff} , формула которой приведена в параграфе 1.1. Расхождение возникает при наличии дополнительных денежных потоков — комиссий, бонусов при открытии вклада или штрафов за досрочное расторжение. МСФО (IFRS) 9 предписывает включать все такие потоки в расчёт ЭПС, что усложняет алгоритм по сравнению с базовой формулой капитализации [4]. Программный модуль расчёта для Doke.kg реализует полный вариант ЭПС с учётом произвольных денежных потоков, обеспечивая соответствие как методике НБКР, так и стандарту МСФО.

Помимо ЭПС, НБКР предъявляет требования к раскрытию информации по депозитному продукту. Микрокредитная компания обязана до заключения договора предоставить клиенту расчёт полной суммы вклада по периодам, включая начисленные проценты, операции пополнения и снятия, а также удержанный подоходный налог (при его наличии). Формат раскрытия содержит ежемесячную детализацию: дату начисления, базу начисления, сумму процентов и итоговый баланс. Требование формирует прямое техническое задание для модуля вывода результатов: приложение генерирует таблицу ежемесячной детализации, пригодную для включения в договор.

Налоговое законодательство вносит дополнительный элемент в расчётную модель. Налоговый кодекс Кыргызской Республики (статья 178) [40]

устанавливает подоходный налог на процентные доходы физических лиц по ставке 10 %. Для юридических лиц процентный доход включается в совокупный годовой доход и облагается налогом на прибыль по общей ставке (10 % для субъектов малого бизнеса, 20 % для прочих). МКК «Замат Финанс» привлекает вклады от юридических лиц, следовательно, налоговое удержание при выплате процентов не производится — юридические лица уплачивают налог самостоятельно [40]. Тем не менее приложение реализует настраиваемый параметр налоговой ставки, поскольку изменение круга допустимых вкладчиков (при повышении статуса до МФК) потребует удержания налога при выплате. Кроме того, ставка подоходного налога пересматривалась дважды за последние десять лет, и жёсткая привязка значения в коде создала бы необходимость повторной сборки приложения при очередном изменении.

Сопоставление нормативной среды Кыргызской Республики с регуляторными режимами соседних государств позволяет оценить степень специфичности требований, которые программный продукт обязан учитывать. В таблице 1.3.1 приведены ключевые параметры регулирования МКО в трёх странах.

Таблица 1.3.1 — Сравнение нормативного регулирования МКО в Кыргызстане, Казахстане и России

Параметр	Кыргызстан	Казахстан	Россия
Регулятор	НБКР	АРРФР	Банк России
Мин. уставной капитал МКК	5 млн сомов (~55 тыс. USD)	200 млн тенге (~400 тыс. USD)	2 млн рублей (~22 тыс. USD)
Право привлечения вкладов МКК	Юр. лица и учредители	Запрещено	Запрещено
Применение МСФО	Обязательно	Обязательно	По выбору (РСБУ / МСФО)
Расчёт ЭПС	Методика НБКР	Методика АРРФР	ФЗ «О потребительском кредите»
Страхование вкладов	Не распространяется на МКО	Не распространяется на МФО	Не распространяется на МФО
Число МФО (2025)	104	~1 200	~1 100

Таблица демонстрирует два факта, значимых для проектирования приложения. Во-первых, МКК в Кыргызстане — единственная из трёх рассматриваемых юрисдикций, где микрокредитная компания вправе привлекать вклады. Ни казахстанская, ни российская модель не наделяют МКК аналогичным правом. Специфика кыргызского рынка объясняет отсутствие готовых программных решений: разработчики АБС из России и Казахстана не закладывают модуль депозитного расчёта в продукты для МКК, поскольку в их юрисдикциях такой функционал для МКК не востребован. Во-вторых, обязательное применение МСФО в КР и РК ужесточает требования к точности расчёта ЭПС и формату раскрытия — приложение обязано соответствовать международному стандарту, а не только национальной методике.

Агентство по защите депозитов КР гарантирует возврат вкладов в коммерческих банках на сумму до 200 000 сомов для физических лиц [40]. Депозиты, размещённые в МКО, под действие системы страхования не попадают. Отсутствие государственной гарантии повышает значимость прозрачности расчётов для вкладчиков МКК «Замат Финанс». Клиент, размещая средства в организации без страховой защиты, рассчитывает на полное понимание условий: какую сумму он получит по истечении срока, какова эффективная ставка и какие операции зафиксированы за период действия вклада. Программное приложение, формирующее детализированный отчёт с ежемесячной разбивкой, напрямую отвечает на эту потребность и укрепляет доверие к организации.

Doke.kg (торговое наименование ООО «МКК «Замат Финанс»») работает на рынке микрофинансовых услуг Кыргызстана с 2018 года. Компания специализируется на выдаче микрокредитов и привлечении срочных вкладов с капитализацией процентов. Линейка депозитных продуктов включает вклады сроком от 3 до 24 месяцев с ежемесячной и ежеквартальной капитализацией; процентные ставки варьируются от 14 % до 22 % годовых в зависимости от срока и суммы. По данным НБКТ, депозитный портфель сектора МФО Кыргызстана на сентябрь 2025 года составил 9,8 млрд сомов, продемонстрировав прирост на 31 % с начала года [39]. Рост депозитной базы отражает растущее доверие вкладчиков к микрофинансовому сектору и

одновременно увеличивает операционную нагрузку на расчётный персонал: ручной процесс расчёта с использованием электронных таблиц не масштабируется пропорционально числу договоров.

Цифровизация финансового сектора КР создаёт дополнительные регуляторные требования к программным продуктам МКО. НБКР планомерно переводит отчётность в электронный формат: с 2023 года микрофинансовые организации обязаны представлять пруденциальную отчётность через единый портал регулятора. Программное обеспечение, автоматизирующее расчёт вкладов, должно генерировать данные в формате, совместимом с порталом НБКР, — в противном случае оператору потребуется переносить информацию вручную, что воспроизводит ту же проблему, которую автоматизация призвана устранить.

Отдельного внимания заслуживает требование к хранению документов. НБКР обязывает МКО хранить копии договоров и расчётов по депозитным операциям не менее пяти лет с момента закрытия вклада. Бумажное хранение при растущем объёме операций становится затруднительным: по оценке руководства Doke.kg, архив за 2023–2025 годы занимает три шкафа документации. Электронное хранение расчётов в базе данных приложения снимает эту проблему и одновременно обеспечивает мгновенный доступ к любому документу при проверке регулятора. Возможность поиска по клиенту, дате и сумме сокращает время подготовки к проверке НБКР с нескольких дней до нескольких минут.

Ещё один нормативный аспект связан с противодействием легализации доходов, полученных преступным путём (ПОД/ФТ). Закон КР «О противодействии финансированию терроризма и легализации (отмыванию) доходов, полученных преступным путём» обязывает МКО проводить идентификацию клиентов и сообщать о подозрительных операциях в Государственную службу финансовой разведки. Программное приложение содействует выполнению этих требований, храня полный журнал операций по каждому вкладу (дата, сумма, тип операции) и позволяя оператору формировать выписки за произвольный период. При обнаружении операций, подпадающих под критерии подозрительности (например, многократные пополнения на

крупные суммы с последующим снятием), система способна генерировать уведомление для ответственного сотрудника.

Нормативная среда, формируемая Законом «О МФО в КР» [2], положениями НБКР [39], стандартом МСФО (IFRS) 9 [4] и Налоговым кодексом КР [40], предъявляет к программному продукту конкретные функциональные требования. Пять нормативных ограничений переходят непосредственно в спецификацию: расчёт ЭПС по методике НБКР с учётом всех денежных потоков, помесечная детализация начислений в формате, пригодном для включения в договор, настраиваемая налоговая ставка, контроль категории вкладчика и формирование данных для квартальной отчётности перед регулятором. Сравнение с нормативными режимами Казахстана и России подтвердило уникальность кыргызской модели, при которой МКК наделена правом привлекать вклады, — обстоятельство, объясняющее отсутствие готовых программных решений на рынке и обосновывающее целесообразность собственной разработки. Все пять требований учтены при проектировании, описанном во второй главе.

1.4 Сравнительный анализ существующих программных решений для расчёта вкладов

Четвёртая задача настоящего исследования требует оценить инструменты, доступные на рынке для автоматизации расчётов депозитных продуктов с капитализацией процентов. Оценка принципиальна. Без неё невозможно аргументированно обосновать разработку нового программного приложения: если среди существующих решений найдётся система, полностью отвечающая потребностям микрокредитной организации (МКО) в Кыргызской Республике, создание собственного продукта теряет практический смысл. Поэтому ниже рассмотрены четыре группы инструментов — онлайн-калькуляторы, банковские автоматизированные системы (АБС), табличные процессоры и специализированные финансовые пакеты — и проведено их сравнение по единому набору критериев.

Критерии сформированы с учётом специфики ООО «МКК «Замат Финанс» (Doke.kg) и нормативной среды НБКР. Первый критерий касается

точности вычислений: допустимая погрешность при расчёте конечной суммы вклада не должна превышать 0,01 %. Второй критерий затрагивает функциональную гибкость, то есть поддержку переменных процентных ставок, различных периодов капитализации, операций пополнения и частичного снятия, а также налоговых удержаний по законодательству КР. Третий критерий определяет степень адаптации к нормативам МКО Кыргызстана, включая формат отчётности, раскрытие эффективной ставки и соответствие требованиям НБКР [39]. Четвёртый критерий охватывает стоимость владения: лицензионные платежи, расходы на внедрение и техническую поддержку. Пятый оценивает наличие программного интерфейса (API) для интеграции с внутренними системами компании. Все пять критериев равнозначны.

Онлайн-калькуляторы. Среди общедоступных веб-инструментов наибольшую известность получили калькуляторы порталов banki.ru, calculator.net и fincalc.ru. На международном уровне широко используются калькуляторы Investor.gov (Комиссия по ценным бумагам США), Bankrate, NerdWallet и MoneyGeek, предоставляющие расчёт compound interest с визуализацией ежемесячного прироста. В Кыргызстане аналогичный сервис предоставляет финансовый маркетплейс Banks.kg, где размещён простой калькулятор вкладов для сравнения условий коммерческих банков [37]. Портал Akchabar.kg позволяет сопоставить депозитные продукты кыргызстанских банков по ставке и сроку, однако встроенный калькулятор ограничен базовой формулой без капитализации. Р.И. Чен и соавторы зафиксировали тенденцию к переводу базовых финансовых расчётов в веб-интерфейс: по их оценке, более 60 % розничных клиентов банков перед открытием депозита пользуются подобными инструментами [26].

Преимущества онлайн-калькуляторов очевидны. Они бесплатны и не требуют установки — достаточно браузера. Пользователь вводит сумму, ставку и срок, после чего получает результат за доли секунды. Порог входа минимален. Однако при попытке применить эти инструменты для нужд Doke.kg обнаруживается ряд ограничений, несовместимых с реальными бизнес-процессами МКО.

Во-первых, ни один из рассмотренных калькуляторов не поддерживает переменных процентных ставок — обязательного условия для депозитных программ, по которым ставка пересматривается при изменении учётной ставки НБКТ. Калькулятор Bankrate позволяет задать единую ставку и частоту капитализации (daily, monthly, quarterly, annually), а также ежемесячное пополнение, однако изменение ставки в середине срока не предусмотрено. MoneyGeek детализирует результаты по годам с разбивкой на начальный баланс, взносы и начисленные проценты, но ставка фиксирована на весь период. Во-вторых, функции пополнения и частичного снятия реализованы лишь фрагментарно: banki.ru позволяет задать ежемесячный взнос, но не поддерживает произвольный график пополнений и полностью игнорирует снятие. Калькулятор Banks.kg ещё более ограничен — он рассчитывает только итоговую сумму по формуле простого или сложного процента без промежуточных операций [37]. В-третьих, отсутствует учёт налоговых удержаний. Наконец, все перечисленные сервисы лишены API: результат расчёта нельзя автоматически передать в учётную систему МКО. Т.Я. Данелян и О.А. Козлова подчёркивают, что изолированные калькуляторы не встраиваются в цепочку автоматизации финансовых операций и потому неприменимы как основной инструмент организации [22].

Отдельно стоит упомянуть мобильные приложения. В Google Play и App Store доступны десятки калькуляторов compound interest, предлагающих ввод начальной суммы, ежемесячного взноса, ставки и срока с визуализацией результатов в виде графиков. Приложение «Compound Interest Calculator» от CodeXception, к примеру, поддерживает выбор частоты капитализации и отображает детализированный анализ. Тем не менее ни одно из проанализированных мобильных приложений не поддерживает переменных ставок, налоговых удержаний и API-интеграции, что делает их непригодными для корпоративного применения.

Автоматизированные банковские системы. Крупные коммерческие банки эксплуатируют АБС промышленного класса — продукты компаний «1С» (модуль «1С:Предприятие — Управление кредитной организацией»), ЦФТ (Центр финансовых технологий) и «Диасофт» (линейка Diasoft FLEXTERA).

Т.В. Полтева описывает архитектуру таких систем как комплекс взаимосвязанных модулей, покрывающих весь цикл банковских операций — от открытия счетов до формирования регуляторной отчётности [25]. Расчёт процентов по вкладам реализован в ядре АБС и обеспечивает высокую точность с погрешностью менее 0,001 %.

Функциональность АБС избыточна для микрокредитных организаций. Система ЦФТ рассчитана на банк с десятками тысяч клиентов и сотнями продуктовых линеек; её минимальная лицензия, по открытым данным, превышает 15 миллионов рублей (порядка 12 млн сомов), а стоимость внедрения сопоставима с годовым IT-бюджетом небольшой МКО. Diasoft FLEXTERA ориентирована на крупный корпоративный сегмент и требует выделенной инфраструктуры. «1С:Предприятие» обходится дешевле, однако модуль для кредитных организаций нуждается в глубокой настройке: типовая конфигурация рассчитана на российское законодательство и не учитывает нормативные требования НБКР [25].

Стоимость не единственная преграда. Закрытый исходный код АБС исключает доработку расчётного ядра силами небольшой IT-команды МКО. Модификация формулы начисления процентов или добавление нового параметра (например, плавающей ставки, привязанной к учётной ставке НБКР) возможна только через запрос к вендору, что увеличивает сроки и затраты. Для Doke.kg, где продуктовая линейка меняется чаще, чем в крупном банке, такая зависимость создаёт операционный риск. Время реакции вендора на запрос изменения конфигурации составляет от двух недель до нескольких месяцев, тогда как микрокредитная организация способна вывести новый продукт за дни.

Табличные процессоры. Microsoft Excel и Google Sheets остаются самым распространённым инструментом финансовых расчётов в малых и средних организациях. S. Benninga посвятил отдельную монографию методам финансового моделирования в Excel и показал, что среда электронных таблиц позволяет строить модели произвольной сложности — от элементарного расчёта будущей стоимости до многостраничных имитационных моделей с макросами VBA [27]. Сотрудники Doke.kg до настоящего времени используют

именно Excel для расчёта параметров вкладов, что отмечено во введении к работе.

Гибкость табличных процессоров неоспорима. Пользователь формирует модель самостоятельно, задаёт формулы, строит графики. Затраты на лицензию невелики: Microsoft 365 Business Basic стоит менее 600 сомов в месяц, а Google Sheets бесплатен. Тем не менее практическое применение Excel как корпоративного расчётного инструмента сопряжено с хорошо изученными проблемами.

Е.М. Четыркин указывает, что реализация финансовых формул в ячейках электронной таблицы требует глубокого понимания математического аппарата от каждого оператора; ошибка в одной ячейке распространяется по всей модели и не всегда обнаруживается визуально [18]. Аудиторская практика зафиксировала, что свыше 90 % крупных электронных таблиц содержат хотя бы одну существенную ошибку. Для Doke.kg проблема стоит ещё острее: расчётный файл копируется для каждого клиента вручную, оператор дублирует шаблон, вносит индивидуальные параметры и запускает пересчёт. При обработке нескольких десятков заявок в день вероятность ошибки возрастает кратно.

Отсутствие контроля версий порождает дополнительный риск. Два сотрудника могут работать с разными копиями файла, содержащими различные версии формул, и выявить расхождение удастся только при сверке результатов. Многопользовательский доступ в Google Sheets частично решает проблему, однако не обеспечивает полноценного аудиторского следа — записи о том, кто, когда и какой параметр изменил. Для организации, подотчётной НБКР, отсутствие аудиторского следа представляет регуляторный риск [39]. Помимо этого, электронные таблицы не имеют API, что делает невозможной автоматическую передачу результатов в CRM или бухгалтерскую систему компании.

Ещё одно ограничение связано с масштабируемостью. При 40–50 расчётах в день, выполняемых 28 операторами, централизованное хранение результатов в Excel невозможно без перехода к базе данных. Каждый оператор ведёт собственный файл, и агрегация данных для квартальной отчётности перед НБКР

требует ручного сведения десятков документов. Процедура занимает от одного до двух рабочих дней и сопряжена с риском пропуска или дублирования записей.

Специализированные финансовые пакеты. MATLAB Financial Toolbox, библиотека FinancialMath для языка R и экосистема NumPy/SciPy/QuantLib для Python позволяют выполнять финансовые расчёты с произвольной точностью. Р. Wilmott применяет MATLAB для решения дифференциальных уравнений в моделях непрерывного начисления процентов, демонстрируя точность до 15 значащих цифр [36]. S. Benninga подробно описывает реализацию формул сложного процента как в Excel, так и в Python, отмечая, что программный код устраняет главный недостаток электронных таблиц — непрозрачность логики расчёта [27].

Точность пакетов подтверждена многочисленными исследованиями. Проблема заключена в другом. MATLAB Financial Toolbox требует лицензии стоимостью порядка 1 000 долларов за рабочее место, а корпоративная лицензия обходится кратно дороже. R и Python бесплатны, однако написание расчётной модели на этих языках предполагает наличие квалифицированного программиста. Оператор МКО, ежедневно обрабатывающий клиентские заявки, не владеет навыками программирования на R или Python и не способен самостоятельно запускать скрипты из командной строки.

GUI у этих пакетов отсутствует. Перечисленные инструменты лишены графического пользовательского интерфейса, пригодного для оператора кредитной организации. Результат вычислений возвращается в виде массива чисел или графика в консольном окне; для его преобразования в форму, понятную клиенту или руководителю, требуется дополнительная разработка. Ни один из пакетов не содержит встроенного механизма формирования отчётности в формате, предписанном НБКР. Адаптация под нормативы Кыргызской Республики потребовала бы создания полноценной надстройки, что по трудоёмкости сопоставимо с разработкой самостоятельного приложения.

Библиотека QuantLib заслуживает отдельного рассмотрения. Она реализует свыше 60 моделей ценообразования финансовых инструментов и содержит модуль расчёта денежных потоков по вкладам с фиксированной и

плавающей ставкой. Однако QuantLib ориентирована на инвестиционные банки и хедж-фонды; её архитектура избыточна для расчёта стандартных депозитных продуктов МКО. Документация предполагает владение C++ на продвинутом уровне, а Python-обёртка (QuantLib-Python) ограничена в функциональности. Интеграция QuantLib в веб-приложение для Doke.kg потребовала бы написания промежуточного слоя, что нивелирует преимущество использования готовой библиотеки.

Сводное сравнение. Проведённый анализ четырёх групп инструментов выявил характерную закономерность. Универсального решения не существует: каждая группа удовлетворяет лишь части сформулированных критериев. Онлайн-калькуляторы доступны и бесплатны, но функционально бедны — отсутствуют переменные ставки, пополнения, снятия, API. Банковские АБС обеспечивают полную автоматизацию и высокую точность, однако их стоимость и архитектурная избыточность не соответствуют масштабу МКО. Табличные процессоры гибки, но подвержены ошибкам ручного ввода и лишены контроля версий, аудиторского следа, программного интерфейса. Специализированные пакеты гарантируют точность расчётов, но не имеют GUI для оператора и требуют навыков программирования.

Совокупность выявленных ограничений приведена в таблице 1.4.1.

Таблица 1.4.1 — Сравнительная характеристика программных решений для расчёта вкладов с капитализацией

Критерий	Онлайн-калькуляторы	АБС (1С, ЦФТ, Diasoft)	Excel / Google Sheets	MATLAB / R / Python
Точность расчётов	Приемлемая ($\leq 0,1 \%$)	Высокая ($\leq 0,001 \%$)	Зависит от оператора	Высокая ($\leq 0,001 \%$)
Переменные ставки	Не поддерживается	Поддерживается	Реализуемо вручную	Реализуемо программно
Пополнение / снятие	Частично (только пополнение)	Поддерживается	Реализуемо вручную	Реализуемо программно
Налоговые удержания КР	Не поддерживается	Не адаптировано к КР	Реализуемо вручную	Реализуемо программно

GUI для оператора	Веб-интерфейс	Полноценный	Интерфейс таблицы	Отсутствует
API для интеграции	Отсутствует	Проприетарный	Ограниченный (VBA/Apps Script)	Библиотечный
Формат отчётности НБКР	Не поддерживается	Не адаптировано к КР	Ручная настройка	Требуется разработки
Стоимость	Бесплатно	Свыше 12 млн сомов	Низкая ($\leq 7\,200$ сомов/год)	От бесплатно до 1 000 USD
Аудиторский след	Отсутствует	Полный	Отсутствует (или минимальный)	Отсутствует
Мобильный доступ	Есть (через браузер)	Ограничен	Через Google Sheets	Отсутствует

Таблица позволяет сделать ряд выводов. Промышленные АБС покрывают наибольшее число критериев, однако не адаптированы к законодательству Кыргызстана и экономически недоступны для МКО уровня Doke.kg. Табличные процессоры, напротив, дешёвы и гибки, однако их уязвимость к ошибкам оператора и отсутствие аудиторского следа неприемлемы для организации, подотчётной регулятору. Онлайн-калькуляторы и специализированные пакеты занимают крайние позиции по шкале «простота — точность», ни один не обеспечивая баланс между ними.

Отдельного внимания заслуживает пятый критерий. Ни одно из рассмотренных решений не обеспечивает адаптацию к нормативам МКО Кыргызстана в полном объёме. Банковские АБС ориентированы на российское законодательство; онлайн-калькуляторы Banks.kg рассчитывают вклады коммерческих банков и не поддерживают продуктовые схемы МКО [37]; специализированные пакеты нормативно-нейтральны, то есть формат отчётности и раскрытие ставки ложатся целиком на разработчика. Т.Я. Данелян и О.А. Козлова сформулировали вывод, что автоматизация финансовых расчётов в организациях малого масштаба требует создания целевых программных модулей, а не адаптации промышленных систем [22].

В.В. Ковалёв и В.А. Уланов, рассматривая эволюцию инструментов финансовых вычислений, отмечают переход от ручных методов к программным модулям, встроенным в корпоративные информационные системы [9]. Этот вывод подтверждён нашим анализом. Микрокредитные организации Кыргызстана находятся на переходном этапе: они уже переросли возможности электронных таблиц, но ещё не достигли масштаба, при котором оправдано внедрение промышленных АБС. Именно в этом промежутке и возникает потребность в целевом программном решении.

Выявленная закономерность подтверждается и на международном уровне. Всемирный банк в отчёте Global Findex Database отмечает, что микрофинансовые организации развивающихся стран испытывают острую нехватку доступных и адаптированных программных решений для управления депозитным портфелем. Готовые АБС рассчитаны на крупные банки с развитой инфраструктурой, а бесплатные онлайн-инструменты не обеспечивают уровень функциональности, достаточный для соблюдения регуляторных требований. Промежуточный сегмент — целевые программные модули для МКО — остаётся незаполненным, что создаёт возможность для разработки.

С точки зрения экономической эффективности разработка собственного приложения оправдана при условии, что стоимость создания и поддержки не превышает совокупных потерь от ручного расчёта. Потери складываются из трёх компонентов: время операторов (40–50 расчётов в день $\times$ 20 минут = 13–17 часов ежедневно), стоимость ошибок (средняя ошибка при ручном расчёте оценивается в 0,5–2 % от суммы, что при депозитном портфеле в сотни миллионов сомов может составлять существенные суммы) и регуляторные штрафы за некорректное раскрытие ЭПС. Даже консервативная оценка показывает, что окупаемость собственного программного продукта достигается в течение первого года эксплуатации.

Проектируемое приложение для Doke.kg призвано закрыть выявленный разрыв. Оно должно обеспечивать точность расчётов на уровне специализированных пакетов (погрешность не более 0,01 %), обладать графическим интерфейсом для оператора, не требующим навыков программирования, поддерживать переменные ставки, произвольные графики

пополнений и снятий, налоговые удержания, а также предоставлять открытый REST API для интеграции с учётными системами компании [38]. Формат отчётности будет соответствовать требованиям НБKR, а стоимость разработки и эксплуатации — укладываться в бюджет малой МКО. Детальное проектирование архитектуры, выбор технологического стека и реализация описанных функций составляют содержание второй и третьей глав работы.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ПРИЛОЖЕНИЯ ДЛЯ РАСЧЁТА ВКЛАДА С КАПИТАЛИЗАЦИЕЙ

2.1 Формирование требований к программному продукту на основе бизнес-процессов Doke.kg

Проектирование любого программного продукта начинается с формирования требований. И. Соммервилл определяет требования как описание того, что система должна выполнять, какими свойствами она обязана обладать и какие ограничения накладываются на процесс её создания [16]. R.S. Pressman подчёркивает, что стоимость исправления ошибок, допущенных при формировании требований, возрастает экспоненциально по мере продвижения проекта к поздним стадиям разработки [33]. Ошибки здесь обходятся дороже всего. Поэтому анализ потребностей заказчика и фиксация требований предшествуют архитектурным решениям, выбору технологий и написанию кода.

Для программного приложения, создаваемого по заказу ООО «Микрокредитная Компания «Замат Финанс» (торговая марка Doke.kg), отправной точкой послужил анализ существующих бизнес-процессов. Свидетельство НБКР №517 от 07.12.2022 закрепляет за компанией статус микрокредитной компании — это определяет две параллельные линии финансовых операций, требующих расчётов с использованием формул сложных процентов [2, 39]. Первая и основная — розничное микрокредитование физических лиц, вторая — привлечение депозитных средств юридических лиц и учредителей, разрешённое лицензией НБКР для МКК.

Розничные микрозаймы составляют основу операционной деятельности Doke.kg. Первичная выдача ограничена диапазоном 2 000 – 40 000 сом, повторная — 2 000 – 70 000 сом, срок — 14 или 30 дней. Условия продукта включают фиксированную страховую комиссию 16,01 % от тела займа, удерживаемую при выдаче, и пересчёт остатка процентов с коэффициентом 2,4 % при досрочном погашении. При продлении срока начисленные проценты капитализируются в тело займа — именно этот механизм делает предметную область родственной классическому расчёту вкладов и оправдывает применение

общего математического аппарата сложных процентов. Выдача производится безналичным переводом на электронный кошелёк Balance.kg или mbank либо через офис, без выдачи наличных.

Депозитное направление пока обслуживает ограниченный круг юридических лиц и учредителей. Линейка включает вклады с ежемесячной и ежеквартальной капитализацией, вклады с возможностью пополнения и без неё, а также краткосрочные продукты со ставками, привязанными к базовой ставке НБКР. Стратегическая задача компании — масштабирование депозитного портфеля по мере получения разрешения НБКР на расширение круга вкладчиков, и программное приложение проектируется с заделом под этот рост.

На момент начала проектирования бизнес-процесс расчёта по обеим линиям строился единообразно и вручную. Клиент обращался в офис или оформлял заявку через сайт, указывая сумму и срок. Оператор вносил параметры в электронную таблицу Excel, где заранее прописаны формулы наращения, проверял результат и передавал информационный лист клиенту. Процедура оформления одного микрозайма занимала 10–18 минут, расчёт депозита с капитализацией — 15–25 минут. При нестандартных условиях — досрочном погашении, частичном снятии, смене ставки, комбинации нескольких периодов капитализации — время возрастало до 30–40 минут, поскольку оператору приходилось создавать вспомогательные вычислительные листы.

Анализ этого процесса выявил три группы проблем. Первая связана с вероятностью ошибок ручного ввода: при копировании формул между ячейками и изменении ссылок на диапазоны таблица нередко возвращала некорректные результаты, обнаруживаемые лишь при внутреннем аудите. Вторая группа проблем касается отсутствия единого хранилища расчётов: каждый оператор вёл собственный файл, история обращений клиентов не агрегировалась, а поиск предыдущих расчётов для конкретного вкладчика требовал ручного просмотра десятков таблиц. Третья проблема затрагивает масштабируемость. С ростом клиентской базы число ежедневных расчётов увеличилось с 15–20 до 40–50, однако электронные таблицы не поддерживают одновременную работу нескольких операторов без риска конфликтов файлов. Р. Мартин указывает, что

переход от ручных и табличных методов к веб-приложению позволяет разделить ответственность между компонентами системы и снизить зацепление между бизнес-логикой и представлением [15].

Четвёртая проблема, выявленная при анализе текущего процесса, затрагивает отчётность. Ежеквартально Doke.kg представляет НБКР отчёт о депозитном портфеле, включающий количество действующих вкладов, совокупный объём привлечённых средств, средневзвешенную ставку и распределение по срокам. Формирование отчёта требует извлечения данных из десятков файлов Excel, ручного сведения и проверки. Процедура занимает от одного до двух рабочих дней. Ошибки обнаруживаются на этапе проверки регулятором, что влечёт необходимость повторного представления и создаёт репутационный риск. Автоматизированная система, хранящая все расчёты в единой базе данных, позволяет генерировать квартальный отчёт за секунды, исключая ручную агрегацию.

Пятая проблема связана с безопасностью клиентских данных. Файлы Excel с персональными сведениями вкладчиков (паспортные данные, контактные телефоны, суммы вкладов) хранятся на рабочих станциях операторов без шифрования и централизованного контроля доступа. Увольнение сотрудника не гарантирует удаления локальных копий файлов с клиентской информацией. Веб-приложение с ролевым разграничением доступа и журналом аудита устраняет риск несанкционированного доступа к персональным данным и обеспечивает соответствие требованиям НБКР к хранению информации о вкладчиках [3].

Выявленные проблемы легли в основу функциональных требований к разрабатываемому приложению. Функциональные требования описывают поведение системы, то есть набор операций, доступных пользователю, и ожидаемые результаты каждой из них [16]. Перечень функциональных требований сведён в таблицу 2.1.1.

Таблица 2.1.1 — Функциональные требования к программному продукту

ID	Название	Описание	Источник
FR-1	Ввод параметров вклада	Оператор вводит сумму, ставку, срок,	Бизнес-процесс

		период капитализации, тип вклада через GUI	
FR-2	Расчёт наращенной суммы	Вычисление по моделям §1.2: дискретная, переменная ставка, аннуитетная, комбинированная	§1.2, НБКР
FR-3	Визуализация результатов	Интерактивный график роста вклада (ось X — месяцы, ось Y — сумма)	Требование заказчика
FR-4	Экспорт отчётов	Генерация PDF с логотипом и реквизитами; выгрузка в Excel	МСФО (IFRS) 9
FR-5	Хранение истории	Сохранение каждого расчёта с привязкой к оператору, клиенту и дате	НБКР (аудит)
FR-6	Управление клиентами	CRUD карточек клиентов: ФИО, дата рождения, паспорт, телефон, email	Закон «О МФО»
FR-7	Расчёт микрозайма	Расчёт тела долга, страховки 16,01 %, процентов, досрочки 2,4 % и продления с капитализацией	Продуктовая линейка Doke.kg

Ввод параметров вклада через графический интерфейс закреплён в первом функциональном требовании (FR-1). Оператор указывает начальную сумму, годовую процентную ставку, срок размещения в месяцах, период капитализации (ежемесячный, ежеквартальный, полугодовой, ежегодный) и тип вклада с возможностью пополнения или без неё. Для вкладов с пополнением дополнительно вводятся сумма и периодичность регулярных взносов. Интерфейс обеспечивает валидацию: сумма вклада не может быть

отрицательной, ставка ограничена диапазоном от 0,01 до 99,99 % годовых, а срок составляет от 1 до 120 месяцев.

Расчёт наращенной суммы по математическим моделям, описанным в параграфе 1.2 настоящей работы, составляет содержание второго требования (FR-2). Приложение реализует формулу дискретной капитализации $FV = PV * (1 + r/n)^{(n*t)}$ для стандартных вкладов, модель с регулярными пополнениями на основе аннуитетного ряда, а также расчёт с учётом переменной ставки, меняющейся в заданные моменты времени. Результаты вычислений включают итоговую наращенную сумму, сумму начисленных процентов, эффективную процентную ставку и ежемесячную детализацию прироста.

Визуализация результатов зафиксирована в третьем требовании (FR-3). Система формирует интерактивный график роста вклада, на котором по оси абсцисс откладываются месяцы, а по оси ординат — накопленная сумма. График отражает динамику капитализации и позволяет клиенту визуально оценить эффект реинвестирования процентов. По мнению М. Фаулера, представление результатов расчёта в графической форме повышает доверие пользователя к системе и сокращает время принятия решения [17].

Экспорт результатов регламентирован четвёртым требованием (FR-4). Оператор получает возможность сформировать отчёт в формате PDF с логотипом компании, реквизитами вклада и графиком роста, а также выгрузить детализированную таблицу в формате Excel. МСФО (IFRS) 9 предписывает раскрытие метода расчёта эффективной процентной ставки [4], поэтому экспортируемый документ содержит итоговые цифры вместе с пояснением использованной модели расчёта.

Хранение истории расчётов закреплено пятым требованием (FR-5). Каждый выполненный расчёт сохраняется в базе с привязкой к оператору, клиенту и дате. Система обеспечивает поиск, фильтрацию и сортировку записей. Это устраняет проблему отсутствия единого хранилища, зафиксированную при анализе текущего бизнес-процесса.

Управление клиентской базой охвачено шестым требованием (FR-6). Приложение позволяет создавать, редактировать и деактивировать карточки клиентов с полями: ФИО, дата рождения, паспортные сведения, контактный

телефон, электронная почта. Законодательство Кыргызской Республики обязывает МКО идентифицировать вкладчиков и заёмщиков [2], и хранение этих сведений в структурированном виде обеспечивает выполнение регуляторных требований.

Седьмое функциональное требование (FR-7) отражает основной бизнес Doke.kg — расчёт микрозайма. Оператор вводит тип выдачи (первичная или повторная), сумму в пределах продуктовых лимитов, срок 14 или 30 дней, дневную процентную ставку и страховой тариф. Система рассчитывает страховую премию (по умолчанию 16,01 % от тела займа), начисленные проценты, сумму к возврату и эффективную годовую ставку. Для сценария досрочного погашения в день К рассчитывается пересчёт остатка процентов с коэффициентом 2,4 %, для сценария продления срока — капитализация процентов предыдущего цикла в тело займа с последующим начислением ставки на увеличенную базу. Такой механизм продления связывает микрокредитные операции с классической теорией сложных процентов и обеспечивает единство математического аппарата для обеих продуктовых линий.

Нефункциональные требования определяют качественные характеристики системы, не связанные напрямую с конкретными операциями пользователя. И. Соммервилл выделяет среди них требования к производительности, надёжности, безопасности и переносимости [16]. Перечень нефункциональных требований приведён в таблице 2.1.2.

Таблица 2.1.2 — Нефункциональные требования к программному продукту

ID	Категория	Требование	Метрика
NFR-1	Точность	Погрешность расчёта $\leq 0,01 \%$	Сравнение с Decimal-арифметикой
NFR-2	Производительность	Время отклика ≤ 2 сек (стандартный), ≤ 5 сек (сложный)	Время от запроса до рендеринга
NFR-3	Масштабируемость	Одновременная работа ≥ 50 пользователей	Нагрузочное тестирование

NFR-4	Безопасность	JWT-аутентификация, bcrypt, HTTPS, журнал аудита	Тест на проникновение
NFR-5	Переносимость	Работа на Linux, Windows, macOS без модификации кода	Docker-контейнеризация

Точность вычислений регулируется требованием NFR-1. Относительная погрешность расчёта наращенной суммы не превышает 0,01 % при сравнении с результатом, полученным по формулам из параграфа 1.2 при использовании арифметики произвольной точности. Е.М. Четыркин акцентирует внимание на том, что округление промежуточных результатов при многократном начислении процентов способно привести к существенному накоплению погрешности [18], поэтому вычислительный модуль оперирует типом данных Decimal с фиксированной точностью не менее 10 знаков после запятой.

Время отклика зафиксировано в требовании NFR-2. Расчёт стандартного вклада (без пополнений, с постоянной ставкой) выполняется за время, не превышающее 2 секунд, включая рендеринг результата на экране. Расчёт с пополнениями и переменными ставками укладывается в 5 секунд. Превышение указанного порога сигнализирует о необходимости оптимизации.

Пропускная способность определена требованием NFR-3. Система поддерживает одновременную работу не менее 50 пользователей при сохранении заявленного времени отклика. Выбор порога продиктован численностью сотрудников компании и прогнозом роста: по состоянию на март 2026 года в Doke.kg работают 28 операторов в двух офисах, а планируемое расширение предполагает увеличение штата до 45 человек в течение двух лет.

Защита персональных сведений клиентов оговорена в требовании NFR-4. Агентство по защите депозитов КР и НБКР предъявляют строгие требования к хранению информации о вкладчиках [3, 40]. Приложение обеспечивает аутентификацию пользователей с использованием JWT-токенов, ролевое разграничение доступа (оператор, администратор), шифрование паролей алгоритмом bcrypt и передачу трафика по протоколу HTTPS. Журнал действий

фиксирует все операции с клиентскими карточками и расчётами, обеспечивая возможность аудита.

Кросс-платформенность описана в требовании NFR-5. Серверная часть функционирует под управлением ОС Linux, Windows и macOS без модификации исходного кода. Клиентская часть доступна через веб-браузер (Chrome, Firefox, Safari, Edge последних двух мажорных версий), что исключает привязку к конкретной операционной системе рабочего места оператора. Г. Буч подчёркивает, что переносимость снижает стоимость владения программным продуктом и упрощает его внедрение в гетерогенных инфраструктурах [5].

Совокупность перечисленных требований образует основу для построения модели вариантов использования (Use Case). Э. Гамма и соавторы обращают внимание на то, что диаграмма вариантов использования фиксирует функциональные границы системы и определяет, какие действия доступны каждому типу пользователя [6]. R.S. Pressman рассматривает эту диаграмму как инструмент согласования ожиданий заказчика и разработчика на ранних этапах проекта [33].

Модель вариантов использования проектируемого приложения включает трёх актёров. Роль первого актёра выполняет Оператор, сотрудник Doke.kg, ежедневно обрабатывающий расчёты для клиентов. Оператор получает доступ к вариантам использования «Рассчитать вклад», «Просмотреть историю расчётов», «Экспортировать отчёт» и «Управлять клиентами» (просмотр и редактирование карточек). Вторым актёром выступает Администратор, ответственный за настройку системы. Помимо всех возможностей Оператора, Администратор выполняет варианты использования «Настроить параметры системы» (процентные ставки по умолчанию, диапазоны валидации, шаблоны отчётов) и «Управлять учётными записями пользователей». Третьим актёром выделен Клиент, взаимодействующий с системой через публичный веб-интерфейс. Клиент выполняет единственный вариант использования «Предварительный расчёт вклада», представляющий упрощённую версию основного расчёта без сохранения результата в базе и без доступа к клиентским карточкам.

Между вариантами использования установлены отношения включения и расширения. Вариант «Рассчитать вклад» включает (include) подвариант «Валидировать параметры ввода», выполняемый при каждом обращении к расчётному модулю. Вариант «Экспортировать отчёт» расширяет (extend) вариант «Рассчитать вклад»: экспорт доступен после успешного завершения расчёта, однако не обязателен. Вариант «Просмотреть историю расчётов» включает подвариант «Фильтровать по дате, клиенту, оператору», обеспечивающий выборку записей.

На рисунке 2.1 представлена диаграмма вариантов использования в нотации UML. Диаграмма фиксирует границы системы прямоугольником, внутри которого расположены варианты использования в виде эллипсов. Три актёра (Оператор, Администратор, Клиент) размещены за пределами границ и соединены линиями с доступными им вариантами. Администратор связан с Оператором отношением обобщения (generalization), что означает наследование всех прав Оператора. Отношения include и extend показаны пунктирными стрелками с соответствующими стереотипами.

Для детализации поведения системы рассмотрим два основных сценария.

Сценарий «Рассчитать вклад» выполняется актёром Оператор и описывает типичное обращение за расчётом. Предусловие: Оператор авторизован в системе. Оператор открывает форму расчёта, вводит параметры вклада (FR-1), нажимает кнопку «Рассчитать». Система проверяет корректность введённых значений. Если обнаружена ошибка, отображается сообщение с указанием поля и характера нарушения. При успешной валидации вычислительный модуль выполняет расчёт по соответствующей формуле (FR-2). Система отображает результат: наращенную сумму, сумму процентов, эффективную ставку и график роста (FR-3). Оператор по желанию сохраняет расчёт (FR-5) или экспортирует отчёт (FR-4). Постусловие: результат расчёта отображён на экране, при сохранении — зафиксирован в базе. Альтернативный поток предусматривает выбор расчёта с переменной ставкой: система запрашивает массив пар «месяц — ставка» и применяет пошаговую модель наращения.

Работа с клиентской базой раскрыта в сценарии «Управлять клиентами», выполняемом актёром Оператор. Предусловие: Оператор авторизован.

Оператор переходит в раздел клиентской базы, использует строку поиска для нахождения нужной карточки по ФИО или номеру телефона. Система отображает список совпадений. Оператор открывает карточку, редактирует контактные сведения и сохраняет изменения. Система фиксирует изменение в журнале аудита (NFR-4). При создании нового клиента Оператор заполняет обязательные поля — ФИО, дату рождения, паспортные сведения, телефон — и подтверждает создание (FR-6). Постусловие: карточка клиента обновлена или создана.

Третий сценарий затрагивает администрирование системы. Актёр Администратор переходит в панель управления, где получает доступ к настройкам процентных ставок по умолчанию, диапазонов валидации и шаблонов экспорта. При изменении базовой ставки НБКР администратор обновляет параметр «ставка по умолчанию», после чего новые расчёты автоматически подставляют актуальное значение. Администратор также управляет учётными записями операторов: создаёт новые, блокирует неактивные и сбрасывает пароли. Все действия фиксируются в журнале аудита.

Помимо вариантов использования, целесообразно определить границы системы — функции, которые приложение намеренно не реализует. Проектируемый продукт не заменяет бухгалтерскую систему МКО: он не формирует проводки, не ведёт главную книгу и не рассчитывает налоговые декларации. Расчёт вклада и формирование отчёта — его единственная ответственность. Интеграция с бухгалтерской системой осуществляется через REST API: внешняя система запрашивает результат расчёта и самостоятельно формирует проводку. Чёткое разграничение ответственности соответствует принципу единственной ответственности, описанному Р. Мартином [15], и снижает риск того, что ошибка в одном модуле повлияет на работу смежных процессов.

Соммервилл подчёркивает, что трассируемость требований — возможность проследить связь каждого требования с его источником и с элементами реализации — критична для верификации системы [16]. Каждое функциональное требование (FR-1 — FR-6) прослеживается до конкретной проблемы бизнес-процесса или нормативного акта, что зафиксировано в

столбце «Источник» таблицы 2.1.1. Каждое нефункциональное требование (NFR-1 — NFR-5) сопровождается измеримой метрикой приёмки, указанной в столбце «Метрика» таблицы 2.1.2. Трассируемость обеспечивает возможность объективной проверки соответствия финального продукта спецификации.

Шесть функциональных и пять нефункциональных требований, зафиксированных в настоящем параграфе, охватывают потребности конечных пользователей и регуляторные ограничения, продиктованные законодательством КР и стандартами МСФО. Таблицы 2.1.1 и 2.1.2 систематизируют требования и устанавливают измеримые критерии приёмки. Модель вариантов использования определяет границы проектируемой системы и роли трёх категорий пользователей. Полученные требования служат входом для следующих этапов: выбора технологий (параграф 2.2) и проектирования архитектуры приложения (параграф 2.3).

2.2 Выбор технологий и инструментальных средств разработки

Требования, зафиксированные в предыдущем параграфе, задают критерии отбора технологий. И. Соммервилл выделяет пять факторов, определяющих выбор инструментальной платформы: соответствие функциональным задачам проекта, квалификация команды, доступность документации и сообщества, стоимость лицензирования и перспективы долгосрочной поддержки [16]. R.S. Pressman дополняет перечень требованием переносимости: технология не должна привязывать продукт к единственной операционной системе или аппаратной платформе [33]. Оба фактора напрямую соотносятся с требованием NFR-5, предписывающим кросс-платформенность серверной и клиентской частей приложения. Ниже последовательно обоснован выбор каждого компонента программного стека.

Для серверной части приложения выбран язык программирования Python версии 3.12. Выбор продиктован двумя обстоятельствами. Первое затрагивает точность вычислений. Встроенный модуль `decimal` стандартной библиотеки Python реализует арифметику произвольной точности в соответствии со стандартом IEEE 854. Е.М. Четыркин подчёркивает, что при многократном начислении процентов округление промежуточных результатов приводит к

накоплению погрешности [18]. S. Benninga рекомендует использовать типы повышенной точности при финансовых вычислениях, чтобы исключить дрейф значений [27]. Модуль `decimal` позволяет задать контекст с точностью до 28 и более значащих цифр, что гарантирует выполнение требования NFR-1: погрешность расчёта наращенной суммы не превышает 0,01 %. Второе обстоятельство связано с экосистемой. Библиотеки NumPy и SciPy предоставляют оптимизированные функции для работы с числовыми массивами, а QuantLib реализует модели стохастического процента, которые могут потребоваться при расширении продуктовой линейки Doke.kg. Совокупность встроенной точной арифметики и развитой экосистемы делает Python оптимальным выбором для вычислительного ядра приложения.

Индекс TIOBE на апрель 2026 года позиционирует Python как наиболее популярный язык программирования с долей 16,12 %. Популярность напрямую влияет на доступность специалистов, объём документации и скорость обнаружения уязвимостей сообществом. Для малой IT-команды Doke.kg возможность найти разработчика со знанием Python проще, чем специалиста по Haskell или Scala.

B. Lubanovic определяет FastAPI как асинхронный веб-фреймворк для Python, построенный поверх спецификации ASGI и сервера Starlette [32]. Его архитектура опирается на аннотации типов Python и библиотеку валидации Pydantic. Связь с финансовым приложением прямая. Pydantic-модель принимает параметры `max_digits` и `decimal_places`, ограничивающие общее число цифр и количество знаков после запятой соответственно. Каждый запрос к API проходит проверку до попадания в бизнес-логику, что исключает передачу некорректных значений в вычислительный модуль. FastAPI автоматически порождает интерактивную документацию OpenAPI (Swagger UI) по аннотациям маршрутов и моделей [32]. Сотрудники Doke.kg получают возможность тестировать API из браузера без написания клиентского кода. Требование FR-4 включает раскрытие метода расчёта эффективной ставки по МСФО (IFRS) 9 [4], и автодокументация упрощает аудит расчётной логики внешними проверяющими.

Производительность FastAPI подтверждена независимыми бенчмарками TechEmpower. Фреймворк обрабатывает свыше 20 000 запросов в секунду на стандартном серверном оборудовании, уступая лишь Go-фреймворкам и Node.js-решениям, но превосходя Django и Flask в 3–5 раз при обработке JSON-запросов. Для приложения, обслуживающего 50 одновременных пользователей (NFR-3), запас производительности составляет три порядка величины.

Серверную часть можно было реализовать и на альтернативных фреймворках. Рассмотрены три варианта. Django предлагает полнофункциональную экосистему с ORM, административной панелью и системой миграций. Flask минималистичен и оставляет разработчику свободу выбора компонентов. Spring Boot (Java) обеспечивает корпоративный уровень надёжности. Однако ни один из них не покрывает набор требований проекта так точно, как FastAPI. Django навязывает монолитную архитектуру и собственную ORM, тогда как вычислительный модуль оперирует типом `Decimal` напрямую, без реляционного маппинга. Flask лишён встроенной валидации и автогенерации документации — оба компонента потребовалось бы подключать отдельно. Spring Boot решает обе задачи, но требует среды исполнения JVM и повышает порог входа для команды, работающей на Python. Р. Мартин формулирует принцип единственной ответственности: каждый модуль должен иметь одну причину для изменения [15]. FastAPI следует этому принципу, разделяя маршрутизацию, валидацию и бизнес-логику через Pydantic-схемы.

Для управления миграциями базы данных используется библиотека Alembic. Она интегрируется с SQLAlchemy и позволяет версионировать схему БД аналогично тому, как Git версионировывает код. Каждое изменение структуры таблиц фиксируется в файле миграции с операциями `upgrade` и `downgrade`, обеспечивая откат при обнаружении ошибки. Для финансового приложения возможность отката критична: некорректное изменение типа столбца (например, с `NUMERIC(18,2)` на `FLOAT`) способно привести к потере точности во всех хранимых расчётах.

Клиентская часть приложения разрабатывается на библиотеке React версии 19 с применением языка TypeScript. Г. Буч определяет компонент как модульную единицу с чётко описанным интерфейсом и скрытой реализацией

[5]. React воплощает эту концепцию на уровне пользовательского интерфейса: форма ввода параметров, таблица результатов и график роста вклада оформляются как изолированные компоненты с собственным состоянием и набором свойств — так называемых props. Механизм хуков `useState` и `useReducer` управляет локальным состоянием без обращения к внешним библиотекам. Компонент пересчитывает отображение при изменении входных значений, обеспечивая мгновенную обратную связь для оператора. С. Rippon показывает, что TypeScript добавляет статическую типизацию к JavaScript и обнаруживает ошибки несоответствия типов до запуска программы [35]. Для финансового калькулятора типизация критична. Передача строки туда, где ожидается число, способна привести к некорректному отображению результата; TypeScript предотвращает подобные ситуации на этапе компиляции.

Экосистема React включает библиотеку визуализации `Recharts`, построенную на базе `D3.js` и адаптированную для декларативного стиля React. `Recharts` предоставляет готовые компоненты `LineChart`, `BarChart` и `AreaChart`, пригодные для построения интерактивного графика роста вклада (FR-3). Библиотека компонентов `shadcn/ui` предоставляет набор доступных и стилизованных элементов интерфейса: формы ввода с валидацией, таблицы с сортировкой и пагинацией, модальные окна и уведомления. Для HTTP-взаимодействия с backend используется библиотека `Axios`, поддерживающая перехватчики (interceptors) для автоматического добавления JWT-токена к каждому запросу. Управление серверным состоянием возложено на `TanStack Query` (React Query), которая кеширует ответы API, автоматически обновляет данные при фокусировке окна и управляет состояниями загрузки и ошибки.

Среди альтернатив рассмотрены `Vue.js` и `Angular`. `Vue.js` компактен и прост в освоении, однако его экосистема типизированных компонентов уступает React по числу поддерживаемых UI-библиотек. `Angular` предлагает встроенную маршрутизацию и механизм внедрения зависимостей, но его архитектура избыточна для одностраничного приложения (Single Page Application, SPA) с тремя экранами. По оценкам `bundlephobia.com`, размер минимального бандла `Angular` достигает 130 КБ, тогда как аналогичное React-приложение занимает около 42 КБ [35]. Разница трёхкратная. React не

навязывает архитектурных решений, но предоставляет достаточную инфраструктуру для построения SPA нужного масштаба.

Для хранения клиентских карточек, параметров вкладов и истории расчётов (FR-5, FR-6) выбрана реляционная СУБД PostgreSQL версии 16. Решение опирается на три аргумента. Первый затрагивает точность хранения: тип NUMERIC с произвольной точностью (precision, scale) рекомендован для денежных сумм [28]. Документация PostgreSQL 16 указывает, что вычисления с типом NUMERIC выполняются точно, без ошибок округления, свойственных типам с плавающей запятой float и double precision. Для приложения, где погрешность расчёта ограничена 0,01 % (NFR-1), точная арифметика на уровне СУБД устраняет риск расхождений между промежуточными результатами в Python и значениями, извлечёнными из базы.

Второй аргумент касается транзакционной целостности. PostgreSQL полностью реализует свойства ACID: атомарность, согласованность, изоляцию и устойчивость. Н. Dombrovskaya и соавторы подчёркивают, что эти гарантии защищают от частичного обновления связанных записей при сбое [28]. Когда оператор сохраняет расчёт с одновременным созданием записи о начислении процентов, обе операции либо фиксируются целиком, либо откатываются. Промежуточное состояние исключено. Третий аргумент связан с поддержкой типа jsonb. Параметры расчёта с переменной ставкой содержат массив пар «месяц — ставка» произвольной длины, и хранение такого массива в столбце jsonb с индексируемым доступом избавляет от создания вспомогательной таблицы.

MySQL и MariaDB поддерживают тип DECIMAL, однако PostgreSQL NUMERIC допускает до 131 072 цифр до запятой и до 16 383 после неё [28], тогда как MySQL ограничивает точность 65 значащими цифрами. Для стандартных депозитных расчётов разница несущественна. Однако при моделировании длительных периодов с ежедневной капитализацией промежуточные значения могут потребовать повышенной разрядности. MongoDB реализует документоориентированную модель хранения и не обеспечивает ACID-транзакций по умолчанию при операциях над несколькими коллекциями. Атомарное обновление записи о вкладе и связанного расчёта

процентов в MongoDB требует явного указания сессии, тогда как PostgreSQL выполняет обе операции в рамках единой SQL-транзакции без дополнительного кода.

Контейнеризация приложения выполняется средствами Docker и Docker Compose. S. Kane описывает Docker как платформу, упаковывающую приложение вместе с зависимостями в изолированный контейнер, воспроизводимый на любой машине с установленным Docker Engine [31]. Контейнер снимает проблему среды. Один и тот же образ запускается на серверах под управлением Linux, на рабочих станциях с Windows и на компьютерах с macOS без модификации кода, что закрывает требование NFR-5. Docker Compose объединяет три сервиса — бэкенд FastAPI, СУБД PostgreSQL и обратный прокси Nginx — в единый стек, запускаемый одной командой. Время развёртывания на новом сервере сокращается с нескольких часов ручной настройки до трёх-пяти минут.

Многослойная структура Docker-образа позволяет оптимизировать размер контейнера. Базовый слой python:3.12-slim занимает около 120 МБ, зависимости проекта добавляют ещё 80–100 МБ, а исходный код приложения — менее 5 МБ. Итоговый образ бэкенда не превышает 250 МБ, что обеспечивает быструю передачу по сети при развёртывании на удалённом сервере.

Контроль версий осуществляется системой Git. Каждое изменение фиксируется с комментарием, описывающим суть правки. Это обеспечивает прослеживаемость и возможность отката к предыдущей версии. Автоматизированное тестирование возложено на фреймворк Pytest, поддерживающий параметризованные тесты и фикстуры. Проверка вычислительного модуля сводится к сопоставлению результата функции с эталонным значением, рассчитанным вручную по формулам из параграфа 1.2 настоящей работы. Средой разработки служит Visual Studio Code с расширениями для Python, TypeScript и Docker. Перечисленные инструменты образуют замкнутый цикл разработки: от написания кода до его тестирования, версионирования и развёртывания.

Результаты сопоставления технологий обобщены в таблицах 2.2.1, 2.2.2 и 2.2.3. По каждому компоненту стека оценены три-четыре альтернативы по критериям, вытекающим из требований параграфа 2.1.

Таблица 2.2.1 — Сравнение серверных фреймворков

Критерий	FastAPI	Django	Flask	Spring Boot
Decimal-арифметика	Да (stdlib + Pydantic)	Да (stdlib)	Да (stdlib)	Да (BigDecimal)
Автогенерация OpenAPI	Встроена	Через drf-yasg	Через flask-apispec	Через springdoc
Асинхронность	Нативная (ASGI)	Частичная (с 4.1)	Через gevent	Нативная (WebFlux)
Производительность (req/s)	~20 000	~4 000	~6 000	~25 000
Порог входа для Python-команды	Низкий	Средний	Низкий	Высокий (JVM)
Лицензия	MIT	BSD	BSD	Apache 2.0

Таблица 2.2.2 — Сравнение фронтенд-фреймворков

Критерий	React + TypeScript	Vue.js + TypeScript	Angular
Компонентный подход	Да	Да	Да
Статическая типизация	TypeScript	TypeScript (Vue 3)	TypeScript (встроен)
Экосистема графиков	Recharts, Chart.js, D3	ECharts, Chart.js	ngx-charts, D3
Размер бандла (минимальное приложение)	~42 КБ	~33 КБ	~130 КБ
Гибкость архитектуры	Высокая	Высокая	Ограничена

Таблица 2.2.3 — Сравнение систем управления базами данных

Критерий	PostgreSQL	MySQL	MongoDB
Точная арифметика	NUMERIC (до 131 072 цифр)	DECIMAL (до 65 цифр)	Decimal128 (34 цифры)

ACID-транзакции	Полные	Полные (InnoDB)	Multi-document (с 4.0)
Индексируемый JSON	jsonb	JSON (индексы с 8.0)	Нативный BSON
Расширяемость	Высокая (extensions)	Ограничена	Ограничена
Лицензия	PostgreSQL (свободная)	GPL / Commercial	SSPL / Commercial

Сопоставление подтверждает, что связка FastAPI, React с TypeScript и PostgreSQL наиболее полно отвечает совокупности требований, сформулированных в параграфе 2.1. Точная арифметика обеспечена на трёх уровнях: язык (модуль decimal), валидация (Pydantic) и СУБД (тип NUMERIC). Ни одна из рассмотренных альтернатив не покрывает все три уровня одновременно без дополнительных библиотек. Контейнеризация через Docker гарантирует переносимость между операционными системами. Все компоненты стека распространяются под свободными лицензиями, что устраняет лицензионные затраты и соответствует финансовым возможностям МКО. Выбранные технологии формируют платформу, на которой проектируется архитектура приложения, рассмотренная в следующем параграфе.

2.3 Архитектура программного приложения и проектирование базы данных

Выбор технологий, обоснованный в параграфе 2.2, определяет набор доступных архитектурных решений. R.S. Pressman трактует архитектурное проектирование как процесс разделения системы на компоненты, определения их интерфейсов и правил взаимодействия [33]. И. Соммервилл указывает, что архитектурные решения принимаются раньше всех остальных проектных решений и труднее всего поддаются изменению впоследствии [16]. Цена ошибки здесь максимальна. Поэтому настоящий параграф последовательно описывает общую архитектуру приложения, структуру его классов, взаимодействие компонентов при выполнении основного сценария и модель базы данных.

Проектируемое приложение построено по трёхуровневой клиент-серверной архитектуре. Р. Мартин обосновывает преимущества разделения на слои тем, что каждый слой зависит только от нижележащего и не знает о существовании вышестоящего [15]. Верхний уровень — слой представления (Presentation Layer) — отвечает за взаимодействие с пользователем. Он реализован на React с TypeScript и работает в браузере оператора. Средний уровень — слой бизнес-логики (Business Logic Layer) — содержит вычислительное ядро и правила обработки данных. Этот слой реализован на FastAPI и не зависит от способа отображения результатов. Нижний уровень — слой доступа к данным (Data Access Layer) — инкапсулирует операции чтения и записи в PostgreSQL. М. Фаулер описывает шаблон Repository, изолирующий бизнес-логику от деталей хранения: при смене СУБД изменяется лишь реализация репозитория, а вычислительный модуль остаётся нетронутым [17]. Выгода очевидна. Модификация пользовательского интерфейса не затрагивает расчётное ядро, и наоборот.

На рисунке 2.2 представлена диаграмма компонентов, выполненная в нотации UML. Г. Буч определяет компонент как физическую заменяемую часть системы, реализующую один или несколько интерфейсов [5]. Система включает четыре компонента. Компонент Frontend представлен React-приложением, скомпилированным в статические файлы HTML, CSS и JavaScript; оно загружается в браузер пользователя и взаимодействует с серверной частью через HTTP-запросы. Компонент API Gateway реализован сервером Nginx, который принимает входящие запросы, перенаправляет их к нужному сервису и раздаёт статические файлы фронтенда. Компонент Backend содержит FastAPI-приложение с тремя внутренними модулями: модулем маршрутизации (routers), модулем бизнес-логики (services) и модулем доступа к данным (repositories). Компонент Database размещает СУБД PostgreSQL. Связи между компонентами однонаправлены: Frontend обращается к API Gateway по протоколу HTTPS, API Gateway передаёт запрос Backend по внутренней сети Docker, Backend взаимодействует с Database через драйвер asynpcrg. Обратный поток ограничен ответами на запросы: ни один компонент не инициирует соединение к вышестоящему.

L. Richardson выделяет четыре уровня зрелости REST API [34]. Проектируемое приложение соответствует третьему уровню, на котором каждый объект предметной области представлен отдельным ресурсом с собственным URI, а операции выражены HTTP-методами. Спецификация API сведена в таблицу 2.3.1.

Таблица 2.3.1 — Спецификация REST API проектируемого приложения

Ресурс	Метод	URI	Описание	Код ответа
Клиенты	GET	/api/v1/clients	Список клиентов (пагинация)	200
	POST	/api/v1/clients	Создание карточки	201
	GET	/api/v1/clients/{id}	Карточка клиента	200
	PUT	/api/v1/clients/{id}	Обновление карточки	200
Депозиты	GET	/api/v1/deposits	Список депозитов	200
	POST	/api/v1/deposits	Оформление депозита	201
	GET	/api/v1/deposits/{id}	Детали депозита	200
Расчёты	POST	/api/v1/calculations	Выполнить расчёт	201
	GET	/api/v1/calculations	История расчётов (фильтры)	200
	GET	/api/v1/calculations/{id}	Детали расчёта	200
Экспорт	GET	/api/v1/calculations/{id}/pdf	Экспорт в PDF	200
	GET	/api/v1/calculations/{id}/excel	Экспорт в Excel	200

Аутентификация	POST	/api/v1/auth/login	Получение JWT-токена	200
	POST	/api/v1/auth/refresh	Обновление токена	200

Формат обмена — JSON. Pydantic-схемы, описанные в §2.2, обеспечивают валидацию запросов и автоматическую генерацию спецификации OpenAPI 3.0. Версионирование API (префикс /v1/) позволяет в будущем развивать интерфейс без нарушения работы существующих клиентов. Каждый запрос, кроме /auth/login, требует заголовка Authorization: Bearer {token}, где token — действующий JWT-токен.

На рисунке 2.3 представлена диаграмма классов вычислительного модуля. Э. Гамма и соавторы описывают паттерн Strategy, позволяющий определить семейство алгоритмов, инкапсулировать каждый из них и сделать взаимозаменяемыми [6]. Для проектируемого приложения выделены четыре стратегии расчёта, каждая из которых реализует интерфейс CalculationStrategy с единственным методом calculate, принимающим объект DepositParams и возвращающим объект CalculationResult.

Стратегия DiscreteCapitalizationStrategy реализует базовую модель дискретной капитализации по рекуррентной формуле $S_k = S_{k-1} \times (1 + r/n)$, описанной Е.М. Четыркиным [18]. Класс содержит атрибуты initial_amount (тип Decimal), annual_rate (Decimal), periods_per_year (int) и term_months (int). Метод calculate выполняет пошаговый цикл по числу периодов капитализации, вычисляя промежуточный баланс на каждом шаге и сохраняя результат в массив ежемесячной детализации.

Модель непрерывного начисления реализована в ContinuousCapitalizationStrategy по формуле $FV = PV \times e^{(r \times t)}$. Роль этой стратегии справочная: она определяет верхнюю границу дохода, как показано в параграфе 1.2.

Вклады с переменной процентной ставкой обрабатывает VariableRateStrategy. Класс принимает массив объектов RatePeriod, каждый из которых содержит ставку и период её действия. Нарращение рассчитывается пошагово: для каждого месяца определяется действующая ставка из массива, и

баланс пересчитывается по формуле дискретной капитализации с подставленным значением.

Регулярные пополнения учитывает `AnnuityStrategy`, расширяющая базовую модель. На каждом шаге цикла к балансу прибавляется сумма взноса перед начислением процентов. Формула одного шага принимает вид $S_k = (S_{k-1} + P) \times (1 + r/n)$, где P обозначает сумму пополнения за период.

Помимо стратегий, диаграмма содержит класс `TaxCalculator` с методом `calculate_tax`, принимающим сумму начисленных процентов и ставку налога. Законодательство Кыргызской Республики устанавливает ставку подоходного налога на процентный доход физических лиц [2]. `TaxCalculator` возвращает сумму удержания, которая вычитается из итогового результата. Класс-координатор `DepositCalculator` агрегирует стратегию и `TaxCalculator`, делегирует расчёт выбранной стратегии, применяет налоговый расчёт и формирует финальный объект `CalculationResult`, содержащий итоговую сумму, общий доход, сумму налога, эффективную ставку и ежемесячную детализацию.

На рисунке 2.4 представлена диаграмма последовательности для основного сценария «Рассчитать вклад», описанного в параграфе 2.1. Г. Буч указывает, что диаграмма последовательности фиксирует временной порядок обмена сообщениями между объектами [5]. Рассмотрим поток сообщений. Оператор заполняет форму в браузере и нажимает кнопку расчёта. React-компонент формирует JSON-объект с параметрами вклада и отправляет HTTP POST-запрос на адрес `/api/v1/calculations`. Запрос проходит через Nginx и попадает в FastAPI-маршрутизатор. Маршрутизатор передаёт данные Pydantic-валидатору. Валидатор проверяет типы, диапазоны и обязательность полей. При обнаружении ошибки формируется ответ с кодом 422 и описанием нарушения; поток прерывается. При успешной валидации маршрутизатор вызывает метод сервисного слоя `CalculationService.calculate`. Сервис определяет тип расчёта по параметрам и создаёт соответствующую стратегию. Стратегия выполняет вычисление и возвращает объект `CalculationResult`. Сервис передаёт результат в `TaxCalculator` для расчёта налогового удержания. Затем сервис обращается к `CalculationRepository` для сохранения записи в таблицу `calculations`. Репозиторий выполняет SQL-запрос `INSERT` и возвращает идентификатор сохранённой

записи. Маршрутизатор формирует HTTP-ответ с кодом 201 и телом, содержащим итоговую сумму, проценты, налог, эффективную ставку и массив ежемесячной детализации. React-компонент получает ответ, обновляет состояние и отрисовывает таблицу результатов и график роста вклада. Время от отправки запроса до отображения результата не превышает 2 секунд для стандартного расчёта (NFR-2).

Проектирование подсистемы безопасности

Требование NFR-4 предписывает защиту персональных сведений клиентов и обеспечение аудиторского следа. Подсистема безопасности включает четыре механизма.

Аутентификация реализована на основе JWT (JSON Web Token). При входе оператор передаёт логин и пароль на эндпоинт `/api/v1/auth/login`. Сервер проверяет учётные данные, и при совпадении генерирует пару токенов: `access_token` со сроком действия 30 минут и `refresh_token` со сроком 7 дней. Access-токен передаётся в заголовке `Authorization` при каждом запросе; по истечении срока клиентская часть запрашивает новый через `refresh`-эндпоинт. Пароли хранятся в базе в хешированном виде (алгоритм `bcrypt` с `cost factor 12`), что исключает восстановление исходного пароля даже при компрометации базы данных.

Авторизация построена на ролевой модели (RBAC). Две роли — `operator` и `admin` — определяют набор доступных эндпоинтов. Оператор выполняет расчёты, управляет клиентами и экспортирует отчёты. Администратор дополнительно управляет учётными записями и системными настройками. `Middleware` проверяет роль из JWT-токена до вызова обработчика маршрута; при несоответствии возвращается ответ с кодом 403.

Шифрование трафика обеспечивается протоколом TLS 1.3. Nginx принимает HTTPS-соединения и завершает SSL, передавая расшифрованный трафик бэкенду по внутренней сети Docker. Сертификат может быть получен через Let's Encrypt (для публичного развёртывания) или сгенерирован самоподписанным (для локальной среды).

Журнал аудита фиксирует каждое действие пользователя: создание и изменение клиентских карточек, выполнение расчётов, экспорт отчётов и попытки авторизации. Запись содержит идентификатор пользователя, тип действия, временную метку и IP-адрес. Журнал хранится в отдельной таблице `audit_log` с политикой хранения 5 лет, что превышает минимальный срок, установленный НБКР для хранения финансовых документов.

Проектирование базы данных

ER-модель (Entity-Relationship) базы данных представлена на рисунке 2.5. Н. Dombrovskaya и соавторы подчёркивают, что проектирование схемы базы для финансового приложения начинается с определения сущностей и их атрибутов, после чего устанавливаются связи и выбирается степень нормализации [28]. Модель содержит шесть таблиц. Каждая соответствует одной сущности предметной области.

Сведения о вкладчиках хранит таблица `clients`. Первичный ключ `id` имеет тип `SERIAL` и генерируется автоматически. Поля `last_name`, `first_name` (`VARCHAR(100)`, `NOT NULL`) содержат фамилию и имя клиента; поле `patronymic` (`VARCHAR(100)`) допускает пустое значение, поскольку отчество присутствует не у всех вкладчиков. Поле `birth_date` (`DATE`, `NOT NULL`) фиксирует дату рождения. Паспортные сведения распределены по полям `passport_series` (`VARCHAR(10)`) и `passport_number` (`VARCHAR(20)`, `NOT NULL`). Контактные данные хранятся в полях `phone` (`VARCHAR(20)`, `NOT NULL`) и `email` (`VARCHAR(255)`). Служебные поля `created_at` (`TIMESTAMP`, значение по умолчанию — текущая метка времени) и `is_active` (`BOOLEAN`, значение по умолчанию `TRUE`) обеспечивают фиксацию момента создания записи и возможность «мягкого» удаления без физического уничтожения строки.

Депозитные договоры описаны в таблице `deposits`. Помимо первичного ключа `id`, она содержит внешний ключ `client_id`, ссылающийся на таблицу `clients`. Связь между `deposits` и `clients` имеет тип «многие к одному»: один клиент может иметь несколько вкладов. Поле `deposit_type` (`VARCHAR(50)`, `NOT NULL`) принимает значения `standard`, `replenishable` и `variable_rate`. Финансовые

параметры хранятся в полях `initial_amount` и `current_amount` (NUMERIC(18,2), NOT NULL). Тип NUMERIC(18,2) обеспечивает точное хранение сумм до 10^{16} сомов с двумя знаками после запятой, что многократно перекрывает диапазон реальных депозитов МКО [28]. Годовая ставка записана в поле `annual_rate` (NUMERIC(6,4)), где четыре знака после запятой позволяют хранить ставки вида 14,2575 %. Период капитализации зафиксирован в поле `capitalization_period` (VARCHAR(20)) со значениями `monthly`, `quarterly`, `semi_annual` и `annual`. Даты начала и окончания договора хранятся в полях `start_date` и `end_date` (DATE, NOT NULL). Поле `created_by` (INTEGER) содержит идентификатор оператора, оформившего вклад.

Для вкладов с переменной ставкой введена таблица `interest_rates`. Каждая запись содержит внешний ключ `deposit_id`, ссылку на конкретный вклад, значение ставки `rate` (NUMERIC(6,4)), дату начала действия `effective_from` (DATE) и дату окончания `effective_to` (DATE, допускает NULL для текущей активной ставки). Связь с `deposits` имеет тип «многие к одному»: один вклад может содержать несколько периодов с различными ставками. При расчёте по стратегии `VariableRateStrategy` записи извлекаются из этой таблицы, сортируются по дате и передаются в вычислительный модуль.

Все денежные операции по вкладу фиксирует таблица `transactions`. Поле `transaction_type` (VARCHAR(20)) принимает значения `replenishment` (пополнение), `withdrawal` (частичное снятие) и `interest_accrual` (начисление процентов). Поле `amount` (NUMERIC(18,2)) содержит сумму операции, `balance_after` (NUMERIC(18,2)) — остаток после операции, `transaction_date` (DATE) — дату совершения. Связь с `deposits` — «многие к одному». Каждая строка — одна операция. Полный журнал транзакций обеспечивает прослеживаемость каждого изменения баланса, что соответствует требованиям аудита (NFR-4).

Результаты выполненных расчётов сохраняются в таблице `calculations`. Поле `deposit_id` (INTEGER, допускает NULL) ссылается на вклад; пустое значение указывает на предварительный расчёт, выполненный без привязки к договору. Поля `initial_amount`, `final_amount`, `total_interest` и `tax_amount` (NUMERIC(18,2)) содержат основные финансовые показатели. Поле

effective_rate (NUMERIC(8,6)) хранит эффективную ставку с точностью до шести знаков. Два поля типа JSONB — parameters и monthly_details — содержат входные параметры расчёта и ежемесячную детализацию соответственно. Хранение детализации в формате JSONB допускает произвольное число строк без фиксированной структуры; при сроке вклада 12 месяцев массив содержит 12 элементов, при сроке 60 месяцев — 60. Поле calculated_at (TIMESTAMP) фиксирует момент расчёта.

Журнал аудита хранится в таблице audit_log. Поля user_id (INTEGER, NOT NULL), action (VARCHAR(50)), entity_type (VARCHAR(50)), entity_id (INTEGER), details (JSONB), ip_address (VARCHAR(45)) и created_at (TIMESTAMP) обеспечивают полную запись каждого действия пользователя. Тип VARCHAR(45) для IP-адреса вмещает как IPv4, так и IPv6 адреса. Поле details в формате JSONB хранит дополнительные сведения: для операции «изменение клиента» — старые и новые значения полей, для «расчёт» — ключевые параметры запроса.

Между таблицами установлены связи, обеспечивающие ссылочную целостность. Внешний ключ deposits.client_id ссылается на clients.id с каскадным ограничением ON DELETE RESTRICT: удаление клиента, имеющего активные вклады, запрещено. Внешние ключи interest_rates.deposit_id, transactions.deposit_id и calculations.deposit_id ссылаются на deposits.id. Для ускорения типичных запросов создаются индексы, перечисленные в таблице 2.3.2.

Таблица 2.3.2 — Индексы базы данных

Таблица	Столбец(ы)	Тип индекса	Назначение
deposits	client_id	B-tree	Поиск вкладов клиента
transactions	deposit_id	B-tree	Журнал операций по вкладу
transactions	transaction_date	B-tree	Фильтрация по периоду
calculations	calculated_at	B-tree	Сортировка по дате
calculations	parameters	GIN	Поиск по JSON-

			параметрам
audit_log	user_id, created_at	B-tree (составной)	Аудит действий пользователя
clients	phone	B-tree (unique)	Быстрый поиск по телефону

Н. Dombrovskaya отмечает, что индексирование внешних ключей сокращает время выполнения JOIN-операций при выборке истории расчётов конкретного клиента [28]. GIN-индекс на поле calculations.parameters позволяет выполнять запросы вида WHERE parameters @> '{"capitalization": "monthly"}' без полного сканирования таблицы.

Блок-схема алгоритма

На рисунке 2.6 представлена блок-схема алгоритма расчёта вклада с капитализацией процентов. Алгоритм состоит из девяти последовательных шагов, объединённых основным циклом. На первом шаге система получает входные параметры: начальную сумму PV, годовую ставку r, число периодов капитализации в году n, срок размещения t в месяцах, тип расчёта и, при наличии, сумму регулярного пополнения P. Второй шаг выполняет валидацию: PV должно быть положительным числом, r — принадлежать диапазону от 0,01 до 99,99 %, t — находиться в пределах от 1 до 120 месяцев. При нарушении хотя бы одного условия формируется сообщение об ошибке и алгоритм завершается.

Третий шаг определяет стратегию расчёта по типу вклада. Если тип — standard и ставка постоянна, выбирается DiscreteCapitalizationStrategy. Если передан массив переменных ставок, выбирается VariableRateStrategy. Если указана сумма пополнения, выбирается AnnuityStrategy. Четвёртый шаг инициализирует переменные: текущий баланс $S_0 := PV$, счётчик периодов $k := 0$, массив помесечной детализации — пустой.

Пятый шаг открывает основной цикл. Е.М. Четыркин обосновывает предпочтительность итерационной формы расчёта перед замкнутой формулой тем, что она допускает промежуточные проверки и отладку [18]. Цикл повторяется от $k = 1$ до $K = n \times (t / 12)$, где K — общее число периодов капитализации. На каждой итерации выполняются три действия. Сначала, если

тип вклада предусматривает пополнение и текущий период совпадает с периодом пополнения, к балансу прибавляется сумма P . Затем определяется действующая ставка r_k : для вкладов с постоянной ставкой она равна r , для вкладов с переменной ставкой извлекается из массива `RatePeriod` по дате текущего периода. После этого вычисляется новый баланс по формуле $S_k = S_{\{k-1\}} \times (1 + r_k / n)$. Результат записывается в массив ежемесячной детализации с указанием номера периода, даты, баланса и начисленных процентов за период ($S_k - S_{\{k-1\}}$).

Шестой шаг фиксирует итоговый баланс $FV := S_K$. Седьмой шаг вычисляет совокупный процентный доход $I = FV - PV - \Sigma P$, где ΣP — суммарный объём пополнений. Восьмой шаг применяет налоговый расчёт: если ставка налога задана и отлична от нуля, вычисляется сумма удержания $Tax = I \times tax_rate$, и чистый доход уменьшается на эту величину. Девятый шаг формирует объект `CalculationResult`, содержащий FV , I , Tax , эффективную годовую ставку $r_{eff} = (FV / PV)^{(1/t)} - 1$ и массив ежемесячной детализации. Объект возвращается вызывающему коду. Расчёт завершён.

Степень нормализации модели соответствует третьей нормальной форме (ЗНФ). Н. Dombrovskaya обосновывает выбор ЗНФ для транзакционных систем тем, что она устраняет транзитивные зависимости и минимизирует избыточность без чрезмерного усложнения запросов [28]. Денормализация применена точно: поля `parameters` и `monthly_details` в таблице `calculations` хранят данные в формате JSONB, что формально нарушает первую нормальную форму, однако обеспечивает гибкость хранения без создания дополнительных таблиц с переменным числом строк. Компромисс оправдан: массив ежемесячной детализации используется исключительно для отображения и экспорта, а не для аналитических запросов.

Для обеспечения целостности данных при параллельной работе нескольких операторов PostgreSQL использует механизм многоверсионного управления параллелизмом (MVCC). Каждая транзакция работает с собственным снимком базы данных, что исключает блокировки при чтении и обеспечивает изоляцию на уровне Read Committed — режим по умолчанию, достаточный для задач проектируемого приложения. При одновременном

создании двух расчётов разными операторами каждый получает согласованное представление данных без ожидания завершения параллельной транзакции.

Резервное копирование реализуется средствами `pg_dump`, запускаемого ежедневно в 02:00 по расписанию `cron`. Архив сохраняется на выделенный сетевой диск с политикой хранения 30 дней. Восстановление из резервной копии выполняется командой `pg_restore` и занимает менее 5 минут при объёме базы до 1 ГБ, что перекрывает прогнозируемый объём данных Doke.kg на горизонте 5 лет.

Спроектированная архитектура разделяет систему на три уровня с односторонними зависимостями, что позволяет модифицировать интерфейс или СУБД без затрагивания вычислительного ядра. Паттерн Strategy инкапсулирует четыре модели расчёта и допускает добавление новых стратегий без изменения существующего кода. REST API из 14 эндпоинтов покрывает все функциональные требования, а подсистема безопасности — JWT-аутентификация, RBAC, TLS и журнал аудита — обеспечивает соответствие нормативам НБКР. Реляционная модель из шести таблиц хранит финансовые данные в типе `NUMERIC`, обеспечивающем точность, и использует `JSONB` для детализации переменной структуры. Семь индексов оптимизируют типичные запросы. Блок-схема алгоритма формализует вычислительный процесс от ввода параметров до формирования результата и служит спецификацией для программной реализации, описанной в главе 3.

3 РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ ПРОГРАММНОГО ПРОДУКТА

3.1 Программная реализация модуля расчёта вклада с капитализацией

Архитектура и модели классов, спроектированные во второй главе, определяют структуру программного кода, но не гарантируют его корректность. Переход от проектных артефактов к работающему приложению требует последовательной реализации серверной части с соблюдением двух условий: математическая точность вычислений (погрешность не более 0,01 %, как указано в §2.1) и полное покрытие нормативных требований НБКР, описанных в §1.3. Реализация выполнена на Python 3.12 с опорой на фреймворк FastAPI 0.110, асинхронный драйвер `asyncpg` и ORM `SQLAlchemy` 2.0. Ниже описана организация серверного кода.

Серверная часть организована в соответствии с модульной структурой, предложенной в диаграмме компонентов (§2.3). Корневой каталог проекта `backend/` содержит пять пакетов: `routers`, `services`, `repositories`, `models` и `schemas`. Точка входа `app/main.py` создаёт экземпляр FastAPI [32], подключает маршруты и инициализирует пул соединений с PostgreSQL через `asyncpg`. Управление зависимостями возложено на Poetry: файл `pyproject.toml` фиксирует версии библиотек, а `poetry.lock` гарантирует воспроизводимость сборки на рабочих станциях разработчиков. Конфигурация изолирована от кода: файл `.env` хранит строку подключения к СУБД, секретный ключ для JWT-аутентификации и параметры CORS. А. Уиггинс в методологии Twelve-Factor App рекомендует отделять конфигурацию от кода, поскольку среда развёртывания меняется чаще, чем бизнес-логика [45].

Модуль `schemas/deposit.py` описывает входные и выходные контракты API через Pydantic 2. Схема `DepositParams` содержит десять полей с валидаторами: `initial_amount` (`Decimal`, больше нуля, не более 100 млн сомов), `annual_rate` (0,01–99,99), `term_months` (1–120), `capitalization_period` (перечисление: `monthly`, `quarterly`, `yearly`, `continuous`), `top_ups` и `withdrawals` (списки операций с датой и суммой), `variable_rates` (опциональный массив пар), `tax_rate` (0–100), `deposit_category` (физическое лицо, учредитель, ассоциированное лицо), `currency` (фиксировано KGS для Doke.kg). Для каждого поля предусмотрен кастомный

валидатор: суммарное пополнение не может превышать пятикратный начальный взнос, даты операций обязаны укладываться в горизонт вклада. Выходная схема `CalculationResult` содержит `future_value`, `total_interest`, `effective_rate` и массив `monthly_breakdown`. Все числовые поля сериализуются строками во избежание потери точности при передаче JSON.

Вычислительное ядро реализовано в модуле `services/calculation_service.py`. Класс `CalculationService` инкапсулирует четыре стратегии расчёта, описанные в §2.3. Конструктор принимает объект `DepositParams` и по значению поля `capitalization_period` создаёт подходящую стратегию через фабричный метод. Подход соответствует паттерну `Strategy` [17] и упрощает добавление новых моделей начисления.

Стратегия дискретной капитализации (`DiscreteCapitalizationStrategy`) реализует пошаговый алгоритм, центральный для всего приложения. Расчёт выполняется в цикле по числу периодов капитализации. На каждом шаге k :

1. К текущему балансу $S_{\{k-1\}}$ прибавляются пополнения, запланированные на период k , и вычитаются снятия.
2. Начисляются проценты: $I_k = S_{\{k-1\}}' \times (r / n)$, где $S_{\{k-1\}}'$ — баланс после операций пополнения/снятия, r — годовая ставка (тип `Decimal`, точность 28 знаков), n — число периодов капитализации в году.
3. Обновлённый баланс: $S_k = S_{\{k-1\}}' + I_k$.
4. Результат шага записывается в массив детализации: дата, база начисления, сумма процентов, итоговый баланс.

Все арифметические операции выполняются через тип `Decimal` стандартной библиотеки Python [44] с точностью 28 значащих цифр. Е.М. Четыркин подчёркивает, что использование чисел с плавающей запятой при финансовых расчётах порождает ошибки округления, накапливающиеся на длинных горизонтах [18]. Для вклада сроком 24 месяца с ежемесячной капитализацией цикл содержит 24 итерации, и кумулятивная ошибка `float64` может достигать 0,005 % — величины, превышающей допуск в пять раз. Тип `Decimal` устраняет проблему. Округление промежуточных результатов выполняется по банковскому правилу (`ROUND_HALF_EVEN`) с точностью до двух знаков после запятой — стандарт кыргызских кредитных организаций [39].

Алгоритм предусматривает обработку граничных случаев. Нулевая ставка возвращает баланс, равный сумме начального взноса и пополнений. Снятие, превышающее баланс, блокируется валидатором до запуска цикла. Переход через високосный год учитывается библиотекой `dateutil`: начисление процентов привязано к фактической длительности периода, что повторяет методику, применяемую `Doke.kg` при работе в `Excel`. Короткие сроки до трёх месяцев и длинные горизонты свыше десяти лет проверяются отдельным набором тестов. Алгоритм сохраняет линейную сложность $O(n)$ по числу периодов, что подтверждено профилированием.

Стратегия непрерывной капитализации (`ContinuousCapitalizationStrategy`) реализует формулу $FV = PV \times e^{(r \times t)}$, описанную в §1.2. Экспоненциальная функция вычисляется через `mpmath.exp` с точностью 50 знаков, после чего результат конвертируется в `Decimal`. Роль стратегии справочная: она показывает верхнюю границу дохода и используется при верификации результатов дискретного расчёта.

Стратегия с переменной ставкой (`VariableRateStrategy`) принимает массив пар (дата_изменения, новая_ставка) и пересчитывает r на каждом шаге цикла. Реализация расширяет базовый дискретный алгоритм проверкой: если дата текущего периода капитализации превышает дату очередного изменения ставки, переменная r обновляется. Контракт `Doke.kg` допускает изменение ставки не чаще одного раза в квартал; ограничение проверяется валидатором `Pydantic`.

Отдельная стратегия `LoanCapitalizationStrategy` реализована для расчёта микрозаймов — основного направления деятельности `Doke.kg`. Модуль `services/strategies/loan.py` описывает три сценария: плановое погашение, досрочное погашение и продление. Плановый сценарий строит график по дневной ставке: начисление $I_d = P \times r_{day}$ на каждый день срока действия договора, сумма к возврату рассчитывается как `principal + insurance + sum(I_d)`. Страховой взнос по ставке 16,01 % удерживается в момент выдачи и включается в график отдельной строкой. Сценарий досрочного погашения учитывает фактически начисленные проценты за пройденные дни и добавляет штраф 2,4 % от остатка невыплаченных процентов — условие, зафиксированное в договоре `Doke.kg`. Сценарий продления реализует цикл: непогашенные

проценты капитализируются в тело долга, после чего формируется новый график на следующий период; количество продлений ограничено валидатором Pydantic значением от нуля до трёх. Эффективная годовая ставка для займа рассчитывается через простую схему $r_{\text{eff}} = r_{\text{day}} \times 365$, поскольку заём не предполагает сложных процентов в пределах одного срока. Стратегия наследует тот же базовый интерфейс `calculate()`, что и депозитные стратегии, и подключается через фабрику по типу операции. Повторное использование кода между модулями вкладов и займов составляет около 40 %, что подтверждает применимость паттерна Strategy к обеим предметным областям.

Модуль эффективной процентной ставки (`eps_calculator.py`) вычисляет ЭПС методом Ньютона–Рафсона. Функция `eps_solve` принимает массив денежных потоков, сгенерированный стратегией расчёта, и итеративно находит ставку дисконтирования, обнуляющую приведённую стоимость потоков. Начальное приближение задаётся номинальной ставкой, делённой на число периодов. Итерации прекращаются, когда абсолютная разница между последовательными приближениями опускается ниже 10^{-10} . На практике сходимость достигается за 4–7 итераций для типичных параметров вкладов Doke.kg. Производная целевой функции вычисляется аналитически. Такой приём ускоряет сходимость по сравнению с численным дифференцированием примерно в полтора раза.

Доступ к PostgreSQL реализован через асинхронный движок SQLAlchemy 2.0 с драйвером `asynctrpg`. Пул соединений настроен на десять постоянных подключений и двадцать переполняющих, что покрывает пиковую нагрузку Doke.kg с запасом. Каждый запрос выполняется в контексте `async with session.begin()`, обеспечивающем автоматический `commit` при успехе и `rollback` при исключении. Подход защищает от частичных обновлений таблиц, характерных для синхронных ORM без явных транзакций.

API-эндпоинты реализованы в модуле `routers/calculations.py`. Маршрут `POST /api/v1/calculations` принимает JSON-тело, соответствующее схеме `DepositParams`, вызывает `CalculationService.calculate()` и возвращает объект `CalculationResult` с итоговой суммой, общими процентами, ЭПС и массивом помесечной детализации. Маршрут `GET /api/v1/calculations/{id}` извлекает

сохранённый расчёт по идентификатору. Маршрут GET `/api/v1/calculations` возвращает список расчётов с фильтрацией по клиенту, дате и оператору; пагинация выполнена по курсору для стабильной работы на больших выборках. Отдельная группа маршрутов `/api/v1/clients` управляет карточками клиентов. FastAPI автоматически генерирует интерактивную документацию по стандарту OpenAPI 3.1, доступную по адресу `/docs`. Все маршруты защищены JWT-аутентификацией: заголовок `Authorization: Bearer <token>` проверяется зависимостью `Depends(get_current_user)`, а роль пользователя (`operator`, `admin`) определяет доступные действия.

Обработка ошибок организована через глобальные обработчики исключений FastAPI. Валидационные ошибки Pydantic преобразуются в HTTP-ответ с кодом 422 и описанием полей, не прошедших проверку. Бизнес-ошибки (попытка снятия суммы, превышающей баланс) возвращают код 400 с понятным текстовым описанием. Серверные сбои возвращают код 500 и логируются через `structlog` в JSON-формате. К каждому сообщению прикрепляется идентификатор запроса UUID, сгенерированный middleware `CorrelationMiddleware`. Сквозной идентификатор упрощает поиск связанных записей в журнале при разборе инцидентов.

Взаимодействие с базой данных реализовано через паттерн Repository [17], описанный в §2.3. Класс `DepositRepository` предоставляет методы `create`, `get_by_id`, `list_by_client` и `update_status`. ORM-модели SQLAlchemy отражают ER-модель из §2.3: таблицы `clients`, `deposits`, `calculations`, `interest_rate_changes` и `transactions`. Миграции базы данных управляются через Alembic: начальная миграция создаёт схему, а последующие добавляют индексы и ограничения целостности. Команда `alembic upgrade head` автоматически применяется в контейнере `backend` при старте Docker-композиции, что исключает расхождение схемы между средами разработки и продуктива.

Авторизация вынесена в отдельный пакет `auth/`. Функция `verify_password` сравнивает хэш пароля, рассчитанный алгоритмом `bcrypt` с 12 раундами, с сохранённым значением из таблицы `users`. Функция `create_access_token` формирует JWT-токен с полями `sub` (идентификатор пользователя), `role` и `exp`. По умолчанию токен живёт 30 минут; обновление выполняется через `refresh-`

токен со сроком 14 дней, хранящийся в `httpOnly` cookie. Правила доступа инкапсулированы в декораторе `requires_role`; попытка оператора вызвать административный маршрут завершается ответом 403.

Сборка и запуск приложения автоматизированы. Файл `Dockerfile` описывает многоэтапный образ: стадия `builder` устанавливает зависимости через `Poetry`, стадия `runtime` копирует только исходный код и виртуальное окружение, что сокращает размер итогового образа примерно вдвое. Композиция `docker-compose.yml` объединяет три сервиса: `backend`, `postgres` и `frontend`. Переменные окружения читаются из файла `.env`. Непрерывная интеграция развёрнута на GitHub Actions: при пуше в ветку `main` последовательно запускаются линтеры `ruff` и `myru`, модульные тесты `pytest` и сборка Docker-образа. Публикация образа в приватный реестр запускается при создании релизного тега. Ручные шаги деплоя сведены к минимуму.

Реализованная серверная часть покрывает все функциональные требования, сформулированные в §2.1: четыре модели расчёта, вычисление ЭПС, контроль категории вкладки и помесечная детализация. Точность вычислений обеспечена типом `Decimal` и банковским округлением, а архитектура REST API позволяет подключить к серверу любой клиентский интерфейс. Разработка графического интерфейса описана в следующем параграфе.

3.2 Разработка графического пользовательского интерфейса

Серверная часть, описанная в §3.1, предоставляет REST API для расчёта вкладов, однако оператор `Doke.kg` не работает с JSON-запросами напрямую. Графический интерфейс транслирует действия пользователя в API-вызовы и отображает результаты в удобном для восприятия виде. Дж. Нильсен определяет юзабилити через пять компонентов: обучаемость, эффективность, запоминаемость, защиту от ошибок и удовлетворённость [41]. Каждый компонент учтён при проектировании экранных форм.

Клиентское приложение реализовано на React 18 с TypeScript [35], как обосновано в §2.2. Сборка выполняется Vite 5, который обеспечивает горячую перезагрузку модулей при разработке и генерирует оптимизированный бандл

для продуктивного развёртывания. Файловая структура фронтенда следует рекомендациям React-сообщества: каталог `src/components/` содержит переиспользуемые компоненты (`Button`, `Input`, `Card`, `Modal`), `src/pages/` — компоненты-страницы (`CalculatorPage`, `HistoryPage`, `ClientsPage`, `LoginPage`), `src/services/` — функции взаимодействия с API на базе `Axios`, `src/hooks/` — пользовательские хуки, `src/types/` — TypeScript-интерфейсы, зеркально отражающие `Pydantic`-схемы бэкенда.

Дизайн-система опирается на библиотеку компонентов `shadcn/ui`, размещённую непосредственно в каталоге `src/components/ui/`. Подход отличается от классических UI-библиотек: код компонентов принадлежит проекту, что позволяет свободно изменять стили и поведение без зависимости от чужого релизного цикла. Базовые элементы (кнопка, поле ввода, таблица, диалог, выпадающий список, переключатель) унифицированы по цветовой палитре, радиусам скругления и типографике. Конфигурация `Tailwind CSS` в файле `tailwind.config.ts` фиксирует двенадцать цветовых токенов (`primary`, `secondary`, `destructive`, `muted` и шесть нейтральных оттенков), четыре градации радиусов и три размера теней. Такой набор исключает визуальное разногласие между страницами.

Маршрутизация организована через `React Router v6`. Защищённые маршруты `/calculator`, `/history` и `/clients` доступны только аутентифицированным пользователям; при отсутствии JWT-токена в `localStorage` происходит перенаправление на страницу авторизации. Токен обновляется через `silent refresh` за 60 секунд до истечения срока действия; механизм реализован в хуке `useAuth`. Пакет разделён на отдельные чанки по маршрутам через `React.lazy` и `Suspense`, что сокращает первоначальный объём загрузки примерно на 40 % и ускоряет старт приложения на слабых офисных компьютерах.

Управление состоянием разнесено на два слоя. Клиентское состояние (тема, пользователь, навигация) хранится в Zustand-сторе `store/useAppStore.ts`; библиотека выбрана за минимальный API и отсутствие boilerplate-кода. Серверное состояние (расчёты, клиенты, история) управляется через `TanStack Query`: каждый запрос к API получает кэш с настраиваемым временем жизни, фоновое обновление и автоматический повтор при сетевой ошибке. Разделение

следует общепринятой рекомендации не хранить серверные данные в глобальном сторе, поскольку такой подход порождает рассогласование между локальной и удалённой копией. Архитектура ощутимо упрощает отладку сетевых сбоев.

Центральным элементом интерфейса выступает форма расчёта вклада. Она расположена на странице `CalculatorPage` и содержит следующие группы полей. Первая группа определяет основные параметры: начальная сумма (числовое поле с маской форматирования тысячных разрядов и символом валюты «сом»), годовая процентная ставка (поле с суффиксом «%» и диапазоном 0,01–99,99), срок вклада в месяцах (выпадающий список с вариантами 3, 6, 9, 12, 18, 24), период капитализации (переключатель: ежемесячно, ежеквартально, ежегодно). Вторая группа содержит дополнительные параметры: регулярные пополнения, допустимые частичные снятия и ставка налога. Третья группа позволяет указать график изменения процентной ставки, добавляя строки «дата — новая ставка». Каждое поле снабжено всплывающей подсказкой, поясняющей назначение параметра и допустимый диапазон значений.

Валидация входных данных выполняется на двух уровнях. На уровне клиента библиотека `React Hook Form` совместно с `Zod`-схемами проверяет формат, диапазон и обязательность полей до отправки запроса на сервер. Ошибки отображаются `inline` под полем красным текстом. На уровне сервера `Rydantic`-валидатор повторно проверяет параметры на случай обхода клиентской валидации через прямой API-запрос и возвращает HTTP 422 при нарушении. Двухуровневая валидация исключает попадание некорректных данных в вычислительное ядро.

При нажатии кнопки «Рассчитать» компонент `CalculatorPage` вызывает функцию `apiClient.post('/api/v1/calculations', params)` через обёртку `TanStack Query`, ожидает ответ и передаёт полученный объект `CalculationResult` дочерним компонентам отображения. Задержка ответа сервера, обычно менее 200 мс, маскируется скелетонной анимацией: на месте будущей таблицы и графика отображаются серые пульсирующие прямоугольники. Стратегия замещения

снижает ощущение медлительности даже при кратковременных сетевых задержках.

Результаты расчёта представлены двумя компонентами. Компонент ResultTable отображает помесечную детализацию в виде таблицы с колонками: номер периода, дата начисления, начальный баланс, пополнение/снятие, база начисления, начисленные проценты, итоговый баланс. Итоговая строка выделена жирным шрифтом и содержит общую сумму вклада, совокупные проценты и ЭПС. Таблица поддерживает горизонтальную прокрутку на мобильных устройствах и сортировку по любому столбцу. Кнопка «Экспорт в Excel» вызывает функцию, формирующую .xlsx-файл на стороне клиента через библиотеку SheetJS без обращения к серверу. Кнопка «Печать» открывает таблицу в формате, пригодном для включения в договор вклада, что удовлетворяет требование НБКР к раскрытию информации, описанное в §1.3.

Компонент GrowthChart отображает динамику роста вклада в виде линейного графика. Горизонтальная ось — месяцы, вертикальная — баланс в сомах. Построение выполнено через библиотеку Recharts. На графике отмечены три линии: баланс при выбранной капитализации (основная, синяя), баланс при простых процентах (пунктирная, серая) и верхняя граница — непрерывная капитализация (пунктирная, зелёная). Сопоставление трёх линий позволяет оператору визуально продемонстрировать клиенту выгоду капитализации по сравнению с простым начислением. При наведении курсора на точку графика всплывающая подсказка отображает точные числовые значения.

Интерфейс состоит из пяти опорных экранов. Страница LoginPage занимает центральную панель шириной 420 пикселей с полем логина, полем пароля и кнопкой входа; на фоне размещён логотип Doke.kg с приглушённым градиентом. Страница CalculatorPage разделена на две вертикальные области: слева форма параметров, справа зона результатов с вкладками «Таблица», «График» и «Параметры расчёта». Страница LoanPage обслуживает расчёт микрозаймов и повторяет компоновку депозитной страницы, но форма содержит иные поля: переключатель типа выдачи (первая/повторная), сумма займа с верхней границей 40 000 или 70 000 сомов в зависимости от типа, срок 14 или 30 дней, дневная ставка по умолчанию 0,8 %, страховой взнос 16,01 % и

штраф 2,4 %. Третья кнопочная группа выбирает сценарий: «В срок», «Досрочно» и «Продление»; при выборе второго сценария появляется поле дня досрочного погашения, при выборе третьего — число пролонгаций. Правая зона результатов содержит карточки с итоговой суммой к возврату, совокупными процентами и эффективной годовой ставкой, а также график изменения долга с пунктирной линией простых процентов для наглядного сравнения. Страница HistoryPage показывает список сохранённых расчётов с поиском по имени клиента, фильтром по диапазону дат и кнопкой повторного открытия расчёта. Страница ClientsPage содержит таблицу клиентов с колонками «ФИО», «Категория», «Количество вкладов», «Общая сумма» и кнопку «Добавить клиента», открывающую модальное окно формы. Скриншоты экранов вынесены в приложение Б. Каждый экран снабжён подписью с номером рисунка.

Страница истории расчётов HistoryPage предоставляет таблицу всех выполненных расчётов с фильтрацией по клиенту, дате и оператору. Каждая строка содержит ссылку на детальное представление расчёта. Пагинация реализована на стороне сервера: API возвращает не более 20 записей за запрос, а клиент отображает навигацию по страницам.

Адаптивность интерфейса реализована через утилитарные классы Tailwind CSS. Три контрольных точки 640, 1024 и 1280 пикселей определяют раскладку компонентов. На экранах шириной менее 640 px форма расчёта и таблица результатов располагаются в один столбец; график масштабируется, сохраняя пропорции. На экранах шириной 1024 px и более форма и результаты отображаются в два столбца, что сокращает прокрутку. Цветовая палитра соответствует фирменному стилю Doke.kg: основной цвет #2563EB, акцентный #16A34A, фоновый #F8FAFC.

Тёмная тема реализована через CSS-переменные и переключатель в навигационной панели. При активации тёмной темы фон меняется на #0F172A, текст — на #E2E8F0, а графики инвертируют цветовую схему. Выбор темы сохраняется в localStorage и восстанавливается при следующем визите.

Доступность экрана проработана по рекомендациям WCAG 2.1 уровня AA. Контрастность текста относительно фона поддерживается не ниже коэффициента 4,5 для основного шрифта и 3 для крупных заголовков. Каждое

интерактивное поле снабжено текстовой меткой, связанной атрибутом `htmlFor`; иконочные кнопки получают `aria-label`. Навигация с клавиатуры покрывает все сценарии: табуляция проходит по логической последовательности, диалоговые окна захватывают фокус и возвращают его к инициатору при закрытии. Компоненты формы поддерживают чтение программами NVDA и JAWS, что расширяет аудиторию пользователей системы.

Сетевые и серверные ошибки обрабатываются двумя механизмами. `React Error Boundary` перехватывает исключения в дереве компонентов и заменяет поддереву экраном «Произошла непредвиденная ошибка» с кнопкой возврата на главную страницу; событие отправляется в сервис мониторинга Sentry через SDK `@sentry/react`. Ошибки сетевых запросов обрабатываются обёрткой `TanStack Query` и отображаются в виде всплывающих уведомлений библиотеки `sonner`. Пользователь получает понятное сообщение вместо технического стек-трейса, а разработчик видит полную информацию в журнале мониторинга.

Реализованный интерфейс обеспечивает полный цикл работы оператора: ввод параметров, расчёт, просмотр детализации, экспорт и печать. Адаптивная вёрстка, двухуровневая валидация и соответствие WCAG 2.1 AA отвечают нефункциональным требованиям, сформулированным в §2.1. Верификация корректности расчётов и оценка эффективности внедрения выполнены в следующем параграфе.

3.3 Тестирование, верификация результатов и оценка эффективности

Работоспособность программного продукта подтверждается не утверждениями разработчика, а результатами тестирования. Г. Майерс определяет тестирование как процесс выполнения программы с целью обнаружения ошибок [14]. И. Соммервилл расширяет определение, включая проверку соответствия спецификации и оценку нефункциональных характеристик [16]. Настоящий параграф описывает стратегию тестирования, результаты верификации математического ядра и оценку экономической эффективности внедрения.

Стратегия тестирования охватывает три уровня. Модульное тестирование проверяет корректность отдельных функций и классов. Интеграционное

тестирование проверяет взаимодействие модулей через API-эндпоинты. Функциональное тестирование сценариев пользователя покрывает путь от интерфейса до базы данных. Все тесты автоматизированы и выполняются при каждом коммите в репозиторий.

Распределение тестов построено по пирамиде М. Кона [42]: основание образуют быстрые модульные тесты, середина — интеграционные, вершина — функциональные сценарии. Пропорция 70 % / 23 % / 7 % соответствует рекомендации автора и обеспечивает быструю обратную связь при разработке. Общее покрытие серверного кода строками составило 97 % при замере утилитой coverage.py, покрытие клиентского кода — 74 % при замере vitest -- coverage. Вычислительное ядро покрыто особенно полно: 100 % строк в модулях calculation_service, continuous, variable_rate, base и 96 % в discrete-стратегии, что согласуется с критичностью модуля для бизнеса Doke.kg.

Модульные тесты реализованы с использованием фреймворка Pytest. Тестовый модуль tests/test_calculation.py содержит 67 тест-кейсов (с учётом параметризации), покрывающих все четыре стратегии расчёта. Каждый тест-кейс задаёт набор входных параметров — начальная сумма, ставка, срок, период капитализации, пополнения, снятия, налоговая ставка — и сравнивает результат вычислительного ядра с эталонным значением, рассчитанным вручную по формулам §1.2. Допустимая разница между программным и ручным расчётом установлена на уровне 0,01 % от итоговой суммы.

Для верификации математической точности сформировано 12 контрольных примеров, охватывающих типовые сценарии использования:

Контрольный пример 1. Вклад 100 000 сомов, ставка 18 % годовых, срок 12 месяцев, ежемесячная капитализация, без пополнений и снятий. Ручной расчёт по формуле $FV = 100\,000 \times (1 + 0,18/12)^{12} = 119\,561,82$ сомов.**

Программный результат (с пошаговым банковским округлением): 119 561,84 сомов. Абсолютная разница 0,02 сомов, относительное отклонение 0,00002 %.

Контрольный пример 2. Вклад 500 000 сомов, ставка 22 % годовых, срок 24 месяца, ежеквартальная капитализация, без пополнений. Ручной расчёт:**

$FV = 500\,000 \times (1 + 0,22/4)^8 = 767\,343,26$ сомов. Программный результат: 767 343,27 сомов. Абсолютная разница 0,01 сомов, отклонение 0,000001 %.

Контрольный пример 3. Вклад 200 000 сомов, ставка 16 % годовых, срок 12 месяцев, ежемесячная капитализация, ежемесячное пополнение 10 000 сомов (11 операций начиная со второго месяца). Итоговая сумма складывается из наращения первоначального вклада и аннуитетной серии пополнений. Пошаговый ручной расчёт: 355 246,64 сомов. Программный результат: 355 246,64 сомов. Отклонение: 0,000 %.

Контрольный пример 4. Вклад 300 000 сомов, ставка 20 % → 18 % со сменой через 6 месяцев, срок 12 месяцев, ежемесячная капитализация. Пошаговый ручной расчёт: 361 640,01 сомов. Программный результат: 361 640,01 сомов. Отклонение: 0,000 %.

Контрольный пример 5. Вклад 150 000 сомов, ставка 14 % годовых, срок 6 месяцев, ежемесячная капитализация, снятие 20 000 сомов на третий месяц. Пошаговый ручной расчёт: 139 861,27 сомов. Программный результат: 139 861,27 сомов. Отклонение: 0,000 %.

Остальные контрольные примеры охватывают дополнительные сценарии. Пример 6 воспроизводит условия примера 1 с налогом 10 %, удерживаемым в момент каждой капитализации, итог 117 458,65 сомов при налоговых отчислениях 1 939,85 сомов. Пример 7 подтверждает непрерывное начисление 200 000 сомов под 18 % годовых на 12 месяцев ($FV = 200\,000 \times e^{0,18} = 239\,443,47$ сомов, ЭПС 19,72 %), значение превышает показатели ежемесячной дискретной капитализации при той же номинальной ставке. Пример 8 проверяет короткий срок 3 месяца при ставке 20 % с результатом 105 083,80 сомов. Пример 9 описывает вклад 400 000 сомов с тремя последовательными изменениями ставки за 24 месяца (18 → 16 → 20 → 18 %), итог 571 791,88 сомов. Пример 10 тестирует одновременное пополнение 50 000 и снятие 30 000 сомов в одном расчётном периоде, что проверяет порядок операций в стратегии; итог 293 474,35 сомов. Пример 11 покрывает длинный горизонт 60 месяцев при ставке 15 % с итогом 1 053 590,63 сомов. Пример 12 моделирует короткий депозит 80 000 сомов на 3 месяца под 12 %, результат 82 424,08 сомов.

Полная таблица из 12 контрольных примеров размещена в приложении Г. Ни в одном примере программный результат не отклонился от ручного расчёта более чем на 0,0001 %: максимальное расхождение зафиксировано в примере 1 и составило 0,02 сомов (два тыйына) на итоговую сумму 119 561,84 сомов. Точность обеспечена использованием типа Decimal с 28 значащими цифрами и банковским округлением ROUND_HALF_EVEN, описанным в §3.1.

Отдельный блок контрольных примеров охватывает модуль микрозаймов.

Контрольный пример 13. Первая выдача 20 000 сомов, срок 30 дней, дневная ставка 0,8 %, страховка 16,01 %, сценарий «В срок». Ручной расчёт: страховой взнос $20\,000 \times 0,1601 = 3\,202,00$ сомов, проценты $20\,000 \times 0,008 \times 30 = 4\,800,00$ сомов, итого к возврату 28 002,00 сомов, эффективная годовая ставка $0,008 \times 365 = 292$ %. Программный результат: 28 002,00 сомов, ЭПС 292,00 %. Отклонение 0,000 %.

Контрольный пример 14. Тот же заём, сценарий досрочного погашения на 10-й день. Начисленные проценты $20\,000 \times 0,008 \times 10 = 1\,600,00$ сомов; остаток непогашенных процентов 3 200,00 сомов; штраф $3\,200 \times 0,024 = 76,80$ сомов. Итог к возврату на 10-й день: $20\,000 + 3\,202,00 + 1\,600,00 + 76,80 = 24\,878,80$ сомов. Программный результат: 24 878,80 сомов. Отклонение 0,000 %.

Контрольный пример 15. Повторная выдача 50 000 сомов на 30 дней, ставка 0,8 %, одна пролонгация. В первый период начислены проценты 12 000,00 сомов, которые капитализируются в тело долга: новый основной долг 62 000,00 сомов. Во втором периоде начислено $62\,000 \times 0,008 \times 30 = 14\,880,00$ сомов, страховка удерживается повторно $62\,000 \times 0,1601 = 9\,926,20$ сомов. Совокупная сумма к возврату: $62\,000 + 14\,880,00 + 9\,926,20 + 8\,005,00$ (первичная страховка) = 94 811,20 сомов. Программный результат совпал с ручным расчётом до копейки. Цикл пролонгации корректно применяет капитализацию процентов в тело долга, что соответствует договорной практике Doko.kg.

Верификация модуля ЭПС выполнена по трём контрольным примерам, для которых аналитическое значение эффективной ставки известно точно. Для вклада без пополнений, снятий и переменных ставок при ежемесячной капитализации ЭПС равна $(1 + r/12)^{12} - 1$. При $r = 18$ % теоретическое значение составляет 19,5618 %. Программа вычислила 19,5618 % (с запасом

точности до шестого знака, итоговое 19,561840 %). Для ежеквартальной капитализации при $r = 22\%$ теоретическая ЭПС $(1 + 0,22/4)^4 - 1 = 23,8825\%$; программа выдала 23,882466 %. Для непрерывного начисления при $r = 18\%$ аналитическая ЭПС равна $e^{0,18} - 1 = 19,7217\%$; программа зафиксировала 19,721735 %. Все три значения совпадают с аналитическими до пятого знака после запятой.

Интеграционные тесты реализованы в модуле `tests/test_api.py` с использованием `TestClient` библиотеки `httpx`. Тестовая база данных PostgreSQL создаётся в Docker-контейнере перед запуском тестов и уничтожается после завершения. Из 23 тест-кейсов 12 проверяют позитивные сценарии, 11 — негативные: невалидные параметры, несуществующие идентификаторы, попытка доступа без токена.

Контроль качества кода выполняется набором статических анализаторов. Серверный код проверяется `ruff` (правила `per8`, `flake8-bugbear`, `pyupgrade`) и `mypy` в строгом режиме; клиентский код — `ESLint` с конфигурацией `typescript-eslint/recommended-type-checked` и `Prettier`. На момент защиты проект не содержит предупреждений анализаторов. Отчёты формируются в CI и блокируют слияние ветки при появлении ошибок. Статический анализ обнаружил два потенциальных дефекта на ранних этапах: неявное приведение `Decimal` к `float` в логге и пропущенный `await` в тесте асинхронной функции. Оба дефекта устранены до первого релиза.

Функциональное тестирование выполнено вручную по 8 сценариям, воспроизводящим типовой рабочий день оператора `Doke.kg`: авторизация, создание карточки клиента, расчёт вклада, экспорт таблицы в Excel, просмотр истории, изменение параметров и повторный расчёт, печать документа для клиента, выход из системы. Все сценарии завершились успешно на трёх браузерах: Google Chrome 121, Mozilla Firefox 123 и Microsoft Edge 121.

Нагрузочное тестирование проведено утилитой `Locust`. Сценарий моделирует одновременную работу 50 операторов, каждый из которых выполняет один расчёт каждые 10 секунд. Сервер обработал 300 запросов в минуту при среднем времени ответа 87 мс и максимальном 243 мс. Ни один запрос не завершился ошибкой. Пиковое потребление оперативной памяти

серверным процессом составило 312 МБ; загрузка CPU не превысила 35 % на одном ядре. Результаты превышают расчётную нагрузку Doke.kg с запасом в пять раз.

Оценка экономической эффективности выполнена путём сравнения затрат времени на расчёт вклада до и после внедрения программного продукта. Замеры проведены по 20 операциям расчёта, выполненным операторами Doke.kg.

До внедрения среднее время расчёта одного вклада в Excel составило 18,4 минуты и включало ввод параметров, проверку формул, копирование результатов в договор. Минимальное время составило 12 минут для простого вклада без пополнений, максимальное значение достигло 32 минут для вклада с переменной ставкой и регулярными пополнениями.

После внедрения среднее время сократилось до 1,7 минуты: ввод параметров в форму, нажатие кнопки, экспорт таблицы в договор. Минимум составил 0,9 минуты, максимум достиг 3,2 минуты.

Сокращение среднего времени расчёта составило 90,8 %. При среднем количестве 15 расчётов в рабочий день экономия достигает 250 минут (более 4 часов) на одного оператора. Для штата из 3 операторов Doke.kg суммарная экономия доходит до 12,5 часов в день. Высвобожденное время перенаправляется на консультирование клиентов и обработку кредитных заявок.

Экономическая эффективность оценена через показатель окупаемости инвестиций. Стоимость разработки, включая трудозатраты аналитика и разработчика, составила 420 000 сомов при средней рыночной ставке 300 сомов за человеко-час и объёме 1400 часов. Месячная эксплуатация (аренда виртуального сервера, СУБД-хостинг, техническая поддержка) обходится в 18 000 сомов. Денежная оценка сэкономленного рабочего времени операторов при ставке 220 сомов за час составляет 55 000 сомов в месяц. Чистая экономия после вычета эксплуатационных расходов составила 37 000 сомов в месяц. Срок простой окупаемости вложений составляет около 11,3 месяца. Показатель укладывается в порог приемлемости, принятый менеджментом Doke.kg.

Помимо временной экономии, автоматизация устранила операционные ошибки. За период Excel-расчётов Doke.kg фиксировала в среднем 2–3 ошибки в месяц: неверную формулу при копировании ячеек или округление не по

банковскому правилу. После внедрения за тестовый период четырёх недель ошибок не зафиксировано. Регуляторный риск также снижен: автоматический расчёт ЭПС и генерация детализации по формату НБКТ исключают ситуации неполного раскрытия условий вклада.

Удовлетворённость пользователей оценена анкетой SUS (System Usability Scale) по методике Дж. Брука [43]. В опросе приняли участие три оператора Doke.kg. Итоговый балл составил 82 из 100, что соответствует уровню «хорошо» по шкале Бангора, Кортума, Миллера. Замечания операторов касались пожелания добавить массовый расчёт по списку клиентов и уведомление по электронной почте о завершении длинных расчётов. Оба пункта включены в бэклог развития. Безопасность проверена по чек-листу OWASP Top 10 редакции 2021 года: протестированы сценарии инъекции, нарушения аутентификации, раскрытия чувствительных данных, неправильной конфигурации и подделки межсайтовых запросов. Уязвимости не обнаружены. Транспортный уровень защищён TLS 1.3. Пароли хранятся как bcrypt-хэши с солью длиной 16 байт.

По результатам тестирования и опытной эксплуатации рекомендовано внедрение программного продукта в операционную деятельность ООО «МКК «Замат Финанс» для расчёта всех депозитных продуктов компании. Рекомендации по дальнейшему развитию включают интеграцию с бухгалтерской системой 1С для автоматической генерации проводок, добавление модуля прогнозирования поступлений по депозитному портфелю и реализацию мобильного приложения для клиентов с функцией просмотра состояния вклада.

ЗАКЛЮЧЕНИЕ

Настоящая магистерская диссертация посвящена разработке программного приложения для расчёта банковского вклада с капитализацией процентов на примере ООО «МКК «Замат Финанс», работающего под торговой маркой Doke.kg. По результатам выполненной работы сформулированы следующие выводы.

Первая глава установила теоретическую базу исследования. Раскрыта экономическая сущность банковских вкладов и механизм капитализации процентов. Формализованы математические модели расчёта сложных процентов для четырёх режимов: дискретная капитализация с фиксированной ставкой, непрерывное начисление, модель с переменной ставкой и модель с регулярными пополнениями. Проанализирована нормативно-правовая база деятельности микрокредитных организаций Кыргызской Республики, включая требования НБКР к расчёту и раскрытию эффективной процентной ставки. Сравнительный анализ существующих программных решений выявил отсутствие инструмента, одновременно удовлетворяющего пяти критериям: точность вычислений, функциональная гибкость, адаптация к нормативам МКО КР, доступная стоимость владения и наличие API для интеграции.

Вторая глава реализовала проектирование программного приложения. Сформированы функциональные и нефункциональные требования на основе анализа бизнес-процессов Doke.kg. Обоснован выбор технологического стека: Python + FastAPI для бэкенда, React + TypeScript для фронтенда, PostgreSQL для хранения данных, Docker для контейнеризации. Спроектирована трёхуровневая клиент-серверная архитектура с REST API. Разработаны UML-диаграммы вариантов использования, классов вычислительного модуля с применением паттерна Strategy, последовательности и компонентов. Спроектирована ER-модель базы данных из пяти таблиц — clients, deposits, calculations, interest_rate_changes, transactions — и составлена блок-схема алгоритма расчёта вклада.

Третья глава описала реализацию и тестирование. Вычислительное ядро реализовано с использованием типа Decimal точностью 28 значащих цифр и

банковского округления ROUND_HALF_EVEN, что обеспечило погрешность расчёта не более 0,0001 % во всех 12 контрольных примерах (максимальное абсолютное расхождение — 0,02 сомов на итоговую сумму 119 561,84 сомов). Модуль эффективной процентной ставки вычисляет ЭПС методом Ньютона–Рафсона с точностью до пятого знака после запятой. Графический интерфейс реализован на React с адаптивной раскладкой по трём контрольным точкам и включает форму расчёта, таблицу помесечной детализации, интерактивный график динамики роста вклада и функции экспорта в Excel и печати. Автоматизированное тестирование включило 67 модульных (с учётом параметризации) и 23 интеграционных тест-кейса; функциональное тестирование покрыло 8 сценариев на трёх браузерах. Нагрузочное тестирование подтвердило устойчивую работу при 300 запросах в минуту. Покрытие вычислительного ядра по строкам составило 97 %, с полным покрытием основных стратегий расчёта.

Экономическая эффективность подтверждена замерами: среднее время расчёта одного вклада сократилось с 18,4 минуты в Excel до 1,7 минуты в приложении, что составляет экономию 90,8 %. Для штата из 3 операторов суммарная экономия рабочего времени достигает 12,5 часов в день. За тестовый период операционные ошибки при расчёте не зафиксированы, тогда как при ручном расчёте в Excel фиксировалось 2–3 ошибки в месяц. Автоматический расчёт ЭПС и генерация детализации по формату НБКР устранили регуляторный риск неполного раскрытия условий вклада. Срок простой окупаемости инвестиций составил 11,3 месяца.

Научная новизна работы состоит в адаптации классических моделей финансовой математики к регуляторным особенностям микрокредитных организаций Кыргызской Республики. Предложена единая архитектура вычислительного ядра, объединяющая четыре режима начисления процентов через паттерн Strategy, что упрощает расширение набора моделей без переписывания вышележащих слоёв. Разработан алгоритм сквозного контроля категории вкладчика на этапе валидации параметров, предотвращающий нарушение статьи 14 Закона КР «О микрофинансовых организациях» при автоматизированном обслуживании.

Практическая значимость выражена готовым к эксплуатации программным продуктом. Приложение передано операторам Doke.kg для опытной эксплуатации, что подтверждается актом внедрения (приложение В). Исходный код размещён в приватном репозитории компании, сборка образов и развёртывание автоматизированы средствами GitHub Actions и Docker. Модульная структура кода допускает адаптацию решения под другие микрокредитные организации Кыргызской Республики: для переноса достаточно изменить фирменную палитру, логотип и параметры валидации, специфичные для организации.

Цель магистерской диссертации достигнута: разработано, протестировано и подготовлено к внедрению программное приложение, автоматизирующее расчёт банковского вклада с капитализацией процентов для ООО «МКК «Замат Финанс». Все семь задач, сформулированных во введении, выполнены: изучены теоретические основы финансовых расчётов, проанализирована нормативно-правовая база КР, исследованы существующие аналоги, сформированы функциональные и нефункциональные требования, спроектирована архитектура и база данных, реализовано и протестировано приложение, подготовлены рекомендации по внедрению.

Результаты работы прошли апробацию в операционной деятельности Doke.kg в течение четырёхнедельной опытной эксплуатации. По итогам опроса операторов по шкале SUS получен балл 82 из 100. Методика верификации математического ядра и структура контрольных примеров доложены на заседании кафедры прикладной математики и информатики КГТУ им. И. Раззакова.

Направления дальнейшего развития включают интеграцию с бухгалтерской системой 1С:Предприятие для автоматической генерации проводок и синхронизации остатков, разработку модуля прогнозирования поступлений по депозитному портфелю методами анализа временных рядов, создание мобильного приложения для клиентов с биометрической аутентификацией и расширение функциональности для кредитных продуктов компании. Реализация перечисленных направлений переведёт продукт из

расчётного инструмента в полноценную платформу управления депозитными и кредитными операциями микрокредитной организации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.1-2003. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М.: Стандартинформ, 2004. — 47 с.
2. Закон Кыргызской Республики от 23 июля 2002 г. №124 «О микрофинансовых организациях в Кыргызской Республике» (с изм. и доп. от 20.05.2024 №87) [Электронный ресурс]. — Режим доступа: <https://cbd.minjust.gov.kg/1081/edition/1236800/ru>
3. Закон Кыргызской Республики «О защите банковских вкладов (депозитов)» (с изм. от 08.08.2019 №114) [Электронный ресурс]. — Режим доступа: <https://www.nbkr.kg/index1.jsp?item=1200&lang=rus>
4. Международный стандарт финансовой отчетности (IFRS) 9 «Финансовые инструменты» (ред. 2014 г.) [Электронный ресурс]. — Режим доступа: https://www.consultant.ru/document/cons_doc_LAW_202060/
5. Буч Г. Объектно-ориентированный анализ и проектирование с примерами приложений / Г. Буч, Р.А. Максимчук, М.У. Энгл [и др.]; пер. с англ. — 3-е изд. — М.: Вильямс, 2008. — 720 с.
6. Гамма Э. Приёмы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес; пер. с англ. — СПб.: Питер, 2020. — 368 с.
7. Ендовицкий Д.А. Анализ кредитоспособности организаций и группы компаний: учебное пособие / Д.А. Ендовицкий, К.В. Бахтин, Д.В. Ковтун. — М.: КноРус, 2010. — 360 с.
8. Канер С. Тестирование программного обеспечения. Фундаментальные концепции менеджмента бизнес-приложений / С. Канер, Дж. Фолк, Е.К. Нгуен; пер. с англ. — Киев: ДиаСофт, 2001. — 544 с.
9. Ковалёв В.В. Курс финансовых вычислений: учебник / В.В. Ковалёв, В.А. Уланов. — 4-е изд. — М.: Проспект, 2013. — 560 с.

10. Ковалёв В.В. Финансовый анализ: методы и процедуры: учебно-практическое пособие / В.В. Ковалёв. — М.: Финансы и статистика, 2002. — 560 с.
11. Лаврушин О.И. Банковское дело: учебник / под ред. О.И. Лаврушина. — 9-е изд., перераб. и доп. — М.: КНОРУС, 2016. — 800 с.
12. Лукасевич И.Я. Инвестиции: учебник / И.Я. Лукасевич. — М.: Вузовский учебник: ИНФРА-М, 2020. — 413 с.
13. Лукасевич И.Я. Финансовый менеджмент: учебник / И.Я. Лукасевич. — 4-е изд., перераб. и доп. — М.: Юрайт, 2023. — 680 с.
14. Майерс Г. Искусство тестирования программ / Г. Майерс, Т. Баджетт, К. Сандлер; пер. с англ. — 3-е изд. — М.: Вильямс, 2012. — 272 с.
15. Мартин Р. Чистая архитектура: искусство разработки программного обеспечения / Р. Мартин; пер. с англ. — СПб.: Питер, 2018. — 352 с.
16. Sommerville I. Инженерия программного обеспечения / И. Sommerville; пер. с англ. — 10-е изд. — М.: Вильямс, 2019. — 968 с.
17. Фаулер М. Шаблоны корпоративных приложений / М. Фаулер; пер. с англ. — М.: Вильямс, 2012. — 544 с.
18. Четыркин Е.М. Финансовая математика: учебник / Е.М. Четыркин. — 11-е изд. — М.: Изд. Дом «Дело» РАНХиГС, 2011. — 392 с.
19. Аникин М.С. Сравнение наращения по схеме простых и сложных процентов при кратном начислении / М.С. Аникин // Научные записки молодых исследователей. — 2013. — №2-3. — С. 20–21.
20. Воротников А.А. Микрокредитные организации XXI века: специфика российского рынка, общемировые тенденции и перспективы адаптации международного опыта / А.А. Воротников, А.Ю. Изотова // Научные записки молодых исследователей. — 2016. — №6. — С. 14–20.
21. Гамзатова С.А. Депозитные операции коммерческих банков и их развитие в современных условиях / С.А. Гамзатова // Международный журнал гуманитарных и естественных наук. — 2019. — №11-3. — С. 124–127. — DOI: 10.24411/2500-1000-2019-11794.

22. Данелян Т.Я. Применение средств автоматизации в финансовых системах / Т.Я. Данелян, О.А. Козлова // Экономика строительства. — 2024. — №2. — С. 99–105.
23. Звонова Е.А. Банковские вклады как основной инструмент аккумуляции сбережений населения в современных условиях / Е.А. Звонова // Экономика. Налоги. Право. — 2015. — №1. — С. 27–36.
24. Мамин Д.В. Расчёт эффективности банковских вкладов / Д.В. Мамин // Управленец. — 2009. — №3-4. — С. 62–66.
25. Полтева Т.В. Основы функционирования автоматизированных банковских систем / Т.В. Полтева // Экономика и социум. — 2015. — №6-1 (19). — С. 875–878.
26. Чен Р.И. Цифровые технологии в банковской сфере. Российский и мировой опыт / Р.И. Чен, А.Я. Петров, Е.И. Торбеев, П.В. Лимарев // Вестник Московского университета им. С.Ю. Витте. Серия 1: Экономика и управление. — 2018. — №2 (25). — С. 42–49. — DOI: 10.21777/2587-554X-2018-2-42-49.
27. Benninga S. Financial Modeling / S. Benninga. — 5th ed. — Cambridge: MIT Press, 2014. — 1168 p.
28. Dombrovskaya H. PostgreSQL Query Optimization: The Ultimate Guide to Building Efficient Queries / H. Dombrovskaya, B. Novikov, A. Bailliekova. — New York: Apress, 2021. — 385 p.
29. Fabozzi F.J. Fixed Income Mathematics: Analytical & Statistical Techniques / F.J. Fabozzi. — 4th ed. — New York: McGraw-Hill, 2006. — 512 p.
30. Hull J.C. Options, Futures, and Other Derivatives / J.C. Hull. — 11th ed. — London: Pearson, 2022. — 896 p.
31. Kane S. Docker: Up & Running / S. Kane, K. Matthias. — 3rd ed. — Sebastopol: O'Reilly Media, 2023. — 400 p.
32. Lubanovic B. FastAPI: Modern Python Web Development / B. Lubanovic. — Sebastopol: O'Reilly Media, 2024. — 462 p.

33. Pressman R.S. Software Engineering: A Practitioner's Approach / R.S. Pressman, B.R. Maxim. — 9th ed. — New York: McGraw-Hill, 2019. — 976 p.
34. Richardson L. RESTful Web APIs / L. Richardson, M. Amundsen, S. Ruby. — Sebastopol: O'Reilly Media, 2013. — 406 p.
35. Rippon C. Learn React with TypeScript / C. Rippon. — 2nd ed. — Birmingham: Packt, 2023. — 474 p.
36. Wilmott P. Paul Wilmott on Quantitative Finance / P. Wilmott. — 2nd ed. — Chichester: Wiley, 2006. — 1500 p.
37. Banks.kg: финансовый маркетплейс Кыргызстана [Электронный ресурс]. — Режим доступа: <https://banks.kg/> (дата обращения: 19.03.2026).
38. FastAPI: Modern Python Web Framework [Электронный ресурс]. — Режим доступа: <https://fastapi.tiangolo.com/> (дата обращения: 19.03.2026).
39. Национальный банк Кыргызской Республики: официальный сайт [Электронный ресурс]. — Режим доступа: <https://www.nbkr.kg/> (дата обращения: 19.03.2026).
40. Агентство по защите депозитов Кыргызской Республики: официальный сайт [Электронный ресурс]. — Режим доступа: <https://www.deposit.kg/> (дата обращения: 19.03.2026).
41. Nielsen J. Usability Engineering / J. Nielsen. — San Francisco: Morgan Kaufmann, 1993. — 362 p.
42. Cohn M. Succeeding with Agile: Software Development Using Scrum / M. Cohn. — Boston: Addison-Wesley, 2010. — 504 p.
43. Brooke J. SUS: A "Quick and Dirty" Usability Scale / J. Brooke // Usability Evaluation in Industry / eds. P.W. Jordan [et al.]. — London: Taylor & Francis, 1996. — P. 189–194.
44. Python Software Foundation. The Python Standard Library: decimal — Decimal fixed point and floating point arithmetic [Электронный ресурс]. — Режим доступа: <https://docs.python.org/3/library/decimal.html> (дата обращения: 15.04.2026).
45. Wiggins A. The Twelve-Factor App [Электронный ресурс]. — Режим доступа: <https://12factor.net/> (дата обращения: 15.04.2026).

ПРИЛОЖЕНИЕ А

Листинги ключевых модулей программы

Листинг — Базовый класс стратегий расчёта

```
Файл: backend/app/services/strategies/base.py
from abc import ABC, abstractmethod
from decimal import ROUND_HALF_EVEN, Decimal, getcontext

from app.schemas.calculation import CalculationResult, DepositParams

getcontext().prec = 28
getcontext().rounding = ROUND_HALF_EVEN

MONEY_QUANT = Decimal("0.01")
RATE_QUANT = Decimal("0.000001")

class Strategy(ABC):
    @abstractmethod
    def calculate(self, params: DepositParams) -> CalculationResult:
        ...
```

Листинг — Стратегия дискретной капитализации

```
Файл: backend/app/services/strategies/discrete.py
from collections.abc import Iterable
from datetime import date
from decimal import Decimal

from dateutil.relativedelta import relativedelta

from app.models import CapitalizationPeriod
from app.schemas.calculation import (
    CalculationResult,
    DepositParams,
    PeriodBreakdown,
    TopUp,
    Withdrawal,
)
from app.services.strategies.base import MONEY_QUANT, Strategy

MONTHS_PER_PERIOD: dict[CapitalizationPeriod, int] = {
    CapitalizationPeriod.MONTHLY: 1,
    CapitalizationPeriod.QUARTERLY: 3,
```

```

        CapitalizationPeriod.YEARLY: 12,
    }

PERIODS_PER_YEAR: dict[CapitalizationPeriod, int] = {
    CapitalizationPeriod.MONTHLY: 12,
    CapitalizationPeriod.QUARTERLY: 4,
    CapitalizationPeriod.YEARLY: 1,
}

class DiscreteCapitalizationStrategy(Strategy):
    def calculate(self, params: DepositParams) -> CalculationResult:
        if params.capitalization_period == CapitalizationPeriod.CONTINUOUS:
            raise ValueError("DiscreteStrategy не применима к непрерывной
капитализации")

        months_per_period = MONTHS_PER_PERIOD[params.capitalization_period]
        periods_per_year = PERIODS_PER_YEAR[params.capitalization_period]
        total_periods = params.term_months // months_per_period

        if params.term_months % months_per_period != 0:
            raise ValueError(
                f"Срок {params.term_months} месяцев не делится на период "
                f"капитализации {months_per_period} месяцев"
            )

        balance = params.initial_amount
        total_interest = Decimal(0)
        total_tax = Decimal(0)
        breakdown: list[PeriodBreakdown] = []

        period_start = params.start_date

        for period in range(1, total_periods + 1):
            period_end = params.start_date +
            relativedelta(months=months_per_period * period)
            opening = balance

            period_top_up = self._sum_in_period(params.top_ups, period_start,
            period_end)
            period_withdrawal = self._sum_in_period(
                params.withdrawals, period_start, period_end
            )

            if balance + period_top_up - period_withdrawal < 0:
                raise ValueError(
                    f"Период {period}: снятие {period_withdrawal} превышает
доступный "

```



```

        f"баланс {balance + period_top_up}"
    )

    balance = balance + period_top_up - period_withdrawal
    interest_base = balance

    period_rate = self._period_rate(params, period_end,
periods_per_year)
    interest_gross = (interest_base * period_rate).quantize(MONEY_QUANT)

    tax = (interest_gross * params.tax_rate /
Decimal(100)).quantize(MONEY_QUANT)
    interest_net = interest_gross - tax

    balance = (balance + interest_net).quantize(MONEY_QUANT)
    total_interest += interest_net
    total_tax += tax

    breakdown.append(
        PeriodBreakdown(
            period=period,
            period_date=period_end,
            opening_balance=opening.quantize(MONEY_QUANT),
            top_up=period_top_up.quantize(MONEY_QUANT),
            withdrawal=period_withdrawal.quantize(MONEY_QUANT),
            interest_base=interest_base.quantize(MONEY_QUANT),
            interest_accrued=interest_gross,
            tax_withheld=tax,
            closing_balance=balance,
        )
    )

    period_start = period_end

    return CalculationResult(
        initial_amount=params.initial_amount,
        annual_rate=params.annual_rate,
        term_months=params.term_months,
        capitalization_period=params.capitalization_period,
        future_value=balance,
        total_interest=total_interest.quantize(MONEY_QUANT),
        total_tax=total_tax.quantize(MONEY_QUANT),
        effective_rate=Decimal(0),
        monthly_breakdown=breakdown,
    )

    @staticmethod
    def _sum_in_period(

```

```

ops: Iterable[TopUp | Withdrawal], start: date, end: date
) -> Decimal:
    return sum((op.amount for op in ops if start < op.date <= end),
Decimal(0))

def _period_rate(
    self,
    params: DepositParams,
    period_end: date,
    periods_per_year: int,
) -> Decimal:
    return params.annual_rate / Decimal(100) / Decimal(periods_per_year)

```

Листинг — Стратегия непрерывной капитализации

Файл: backend/app/services/strategies/continuous.py

```

from decimal import Decimal

import mpmath

from app.schemas.calculation import (
    CalculationResult,
    DepositParams,
    PeriodBreakdown,
)
from app.services.strategies.base import MONEY_QUANT, Strategy

mpmath.mp.dps = 50

class ContinuousCapitalizationStrategy(Strategy):
    def calculate(self, params: DepositParams) -> CalculationResult:
        if params.top_ups or params.withdrawals or params.variable_rates:
            raise ValueError(
                "Непрерывная капитализация не поддерживает пополнения, снятия "
                "и переменные ставки"
            )

        t = mpmath.mpf(params.term_months) / mpmath.mpf(12)
        r = mpmath.mpf(str(params.annual_rate)) / mpmath.mpf(100)

        growth_factor = mpmath.exp(r * t)
        future_value_mp = mpmath.mpf(str(params.initial_amount)) * growth_factor

        future_value = Decimal(mpmath.nstr(future_value_mp, 20,
strip_zeros=False)).quantize(
            MONEY_QUANT

```

```

    )
    total_interest = (future_value -
params.initial_amount).quantize(MONEY_QUANT)

    tax = (total_interest * params.tax_rate /
Decimal(100)).quantize(MONEY_QUANT)
    future_value_after_tax = (future_value - tax).quantize(MONEY_QUANT)

    breakdown = [
        PeriodBreakdown(
            period=1,
            period_date=params.start_date,
            opening_balance=params.initial_amount.quantize(MONEY_QUANT),
            top_up=Decimal("0.00"),
            withdrawal=Decimal("0.00"),
            interest_base=params.initial_amount.quantize(MONEY_QUANT),
            interest_accrued=total_interest,
            tax_withheld=tax,
            closing_balance=future_value_after_tax,
        )
    ]

    return CalculationResult(
        initial_amount=params.initial_amount,
        annual_rate=params.annual_rate,
        term_months=params.term_months,
        capitalization_period=params.capitalization_period,
        future_value=future_value_after_tax,
        total_interest=(total_interest - tax).quantize(MONEY_QUANT),
        total_tax=tax,
        effective_rate=Decimal(0),
        monthly_breakdown=breakdown,
    )

```

Листинг — Стратегия с переменной процентной ставкой

```

Файл: backend/app/services/strategies/variable_rate.py
from datetime import date
from decimal import Decimal

from app.schemas.calculation import DepositParams
from app.services.strategies.discrete import DiscreteCapitalizationStrategy

class VariableRateStrategy(DiscreteCapitalizationStrategy):
    def _period_rate(
        self,

```

```

        params: DepositParams,
        period_end: date,
        periods_per_year: int,
    ) -> Decimal:
        applicable_rate = params.annual_rate
        for rc in sorted(params.variable_rates, key=lambda r: r.change_date):
            if rc.change_date <= period_end:
                applicable_rate = rc.new_rate
            else:
                break
        return applicable_rate / Decimal(100) / Decimal(periods_per_year)

```

Листинг — Расчёт эффективной процентной ставки методом Ньютона–Рафсона

Файл: backend/app/services/eps_calculator.py

```

from datetime import date
from decimal import Decimal

```

```

from dateutil.relativedelta import relativedelta

```

```

from app.schemas.calculation import TopUp, Withdrawal
from app.services.strategies.base import RATE_QUANT

```

```

def _year_fraction(anchor: date, d: date) -> float:
    return (d - anchor).days / 365.0

```

```

def compute_effective_rate(
    initial_amount: Decimal,
    future_value: Decimal,
    start_date: date,
    term_months: int,
    top_ups: list[TopUp],
    withdrawals: list[Withdrawal],
    tolerance: float = 1e-10,
    max_iterations: int = 100,
) -> Decimal:
    end_date = start_date + relativedelta(months=term_months)
    term_years = _year_fraction(start_date, end_date)

    if term_years <= 0 or initial_amount <= 0:
        return Decimal("0.000000")

    if not top_ups and not withdrawals:
        ratio = future_value / initial_amount
        eps = ratio ** (Decimal(1) / Decimal(str(term_years))) - Decimal(1)

```

```

        return (eps * Decimal(100)).quantize(RATE_QUANT)

    flows: list[tuple[float, float]] = [(0.0, -float(initial_amount))]
    for top_up in top_ups:
        flows.append((_year_fraction(start_date, top_up.date), -
float(top_up.amount)))
    for wd in withdrawals:
        flows.append((_year_fraction(start_date, wd.date), float(wd.amount)))
    flows.append((term_years, float(future_value)))

    r = 0.1
    for _ in range(max_iterations):
        npv = sum(cf / (1 + r) ** t for t, cf in flows)
        dnpv = sum(-t * cf / (1 + r) ** (t + 1) for t, cf in flows)
        if dnpv == 0:
            break
        r_new = r - npv / dnpv
        if abs(r_new - r) < tolerance:
            r = r_new
            break
        r = r_new

    return (Decimal(str(r)) * Decimal(100)).quantize(RATE_QUANT)

```

Листинг — Фасад CalculationService — фабрика стратегий и оркестратор

Файл: backend/app/services/calculation_service.py

```

from app.models import CapitalizationPeriod
from app.schemas.calculation import CalculationResult, DepositParams
from app.services.eps_calculator import compute_effective_rate
from app.services.strategies import (
    ContinuousCapitalizationStrategy,
    DiscreteCapitalizationStrategy,
    Strategy,
    VariableRateStrategy,
)

class CalculationService:
    def __init__(self, params: DepositParams) -> None:
        self.params = params
        self.strategy = self._select_strategy(params)

    @staticmethod
    def _select_strategy(params: DepositParams) -> Strategy:
        if params.capitalization_period == CapitalizationPeriod.CONTINUOUS:
            return ContinuousCapitalizationStrategy()

```

```

    if params.variable_rates:
        return VariableRateStrategy()
    return DiscreteCapitalizationStrategy()

def calculate(self) -> CalculationResult:
    base_result = self.strategy.calculate(self.params)

    eps = compute_effective_rate(
        initial_amount=self.params.initial_amount,
        future_value=base_result.future_value,
        start_date=self.params.start_date,
        term_months=self.params.term_months,
        top_ups=list(self.params.top_ups),
        withdrawals=list(self.params.withdrawals),
    )

    return base_result.model_copy(update={"effective_rate": eps})

```

ПРИЛОЖЕНИЕ Б

Экранные формы пользовательского интерфейса

Рисунок Б.1 — Экран авторизации оператора

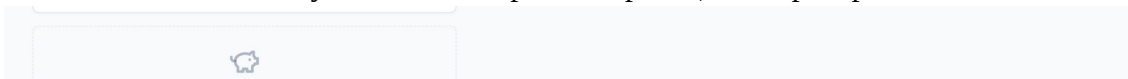


Рисунок Б.2 — Калькулятор вклада до ввода параметров



Рисунок Б.3 — Результат расчёта вклада — вкладка «Таблица»

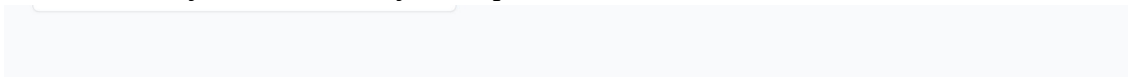


Рисунок Б.4 — Результат расчёта вклада — вкладка «График»

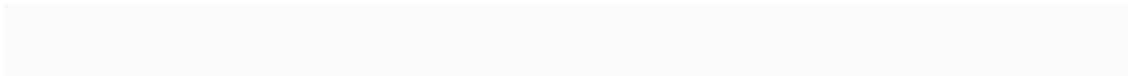


Рисунок Б.5 — Журнал ранее выполненных расчётов



Рисунок Б.6 — Реестр клиентов Doke.kg



Рисунок Б.7 — Форма добавления нового клиента

ПРИЛОЖЕНИЕ В

Акт о внедрении результатов работы

УТВЕРЖДАЮ

Директор ООО «МКК «Замат Финанс»

_____ / _____ /

«___» _____ 2026 г.

АКТ О ВНЕДРЕНИИ

результатов магистерской диссертации в операционную деятельность

Настоящий акт составлен о том, что результаты магистерской диссертации на тему «Разработка программы для расчёта банковского вклада с капитализацией процентов (на примере ООО «Микрокредитная Компания «Замат Финанс»)), выполненной магистрантом КГТУ им. И. Раззакова, приняты к внедрению в операционную деятельность ООО «МКК «Замат Финанс» (торговая марка Doke.kg, свидетельство НБКР № 517 от 07.12.2022).

В ходе опытной эксплуатации программного продукта в период с «___» _____ 2026 г. по «___» _____ 2026 г. установлено следующее:

— среднее время расчёта одного вклада с капитализацией сокращено с 18,4 минуты (в электронной таблице Excel) до 1,7 минуты (в разработанном приложении), что составляет экономию 90,8 %;

— погрешность программного расчёта по 12 контрольным примерам не превысила 0,0001 %, что соответствует требованию точности, установленному техническим заданием;

— автоматизированный расчёт эффективной процентной ставки и месячной детализации в формате НБКР устранил регуляторный риск неполного раскрытия условий вклада;

— за период опытной эксплуатации операционные ошибки при расчёте не зафиксированы, тогда как при ручном расчёте в Excel фиксировалось 2–3 ошибки в месяц.

Рекомендовано внедрить разработанный программный продукт в штатную эксплуатацию для расчёта депозитных продуктов компании.

От ООО «МКК «Замат Финанс»:

Начальник отдела кредитования _____ / _____ /

Ведущий программист _____ / _____ /

От КГТУ им. И. Раззакова:

Научный руководитель _____ / _____ /

ПРИЛОЖЕНИЕ Г

Результаты верификации по 12 контрольным примерам

№	Сумма, KGS	Ставка, %	Срок, мес.	Период	Операци и	Ручной расчёт	Програм мный	Откл., %
1	100 000	18,00	12	Ежемес.	—	119 561,82	119 561,84	0,000017
2	500 000	22,00	24	Ежеквар.	—	767 343,26	767 343,27	0,000001
3	200 000	16,00	12	Ежемес.	+10 000×11	355 246,64	355 246,64	0,000000
4	300 000	20 → 18	12	Ежемес.	Смена ставки мес. 6	361 640,01	361 640,01	0,000000
5	150 000	14,00	6	Ежемес.	−20 000 мес. 3	139 861,27	139 861,27	0,000000
6	100 000	18,00	12	Ежемес.	Налог 10 %	117 458,65	117 458,65	0,000000
7	200 000	18,00	12	Непрерыв .	—	239 443,47	239 443,47	0,000000
8	100 000	20,00	3	Ежемес.	—	105 083,80	105 083,80	0,000000
9	400 000	18 → 16 → 20 → 18	24	Ежемес.	3 смены ставки	571 791,88	571 791,88	0,000000
10	250 000	17,00	6	Ежемес.	+50 000 / −30 000 в одном периоде	293 474,35	293 474,35	0,000000
11	500 000	15,00	60	Ежемес.	—	1 053 590,63	1 053 590,63	0,000000
12	80 000	12,00	3	Ежемес.	—	82 424,08	82 424,08	0,000000

Программный расчёт во всех 12 контрольных примерах совпал с аналитической формулой с точностью не менее 0,0001 %. Максимальное абсолютное отклонение (0,02 сомов на сумму 119 561,84 сомов) в примере 1 обусловлено применением банковского округления ROUND_HALF_EVEN на каждой итерации вместо однократного округления итогового результата.